



République Algérienne Démocratique et Populaire
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
Université des Sciences et de la Technologie Houari Boumediene
Faculté d'Informatique
Département Intelligence Artificielle et Sciences des Données (IA&SD)



Mémoire de Licence

Filière : Informatique

Spécialité : Informatique Académique

Thème :

**Conception et développement d'une application médicale
intelligente pour le suivi et l'assistance des patients**

Encadré par :

Mme MAHANI Aouatef

Proposé et réalisé par :

TERKI Rayane
BOUKERROUM Malik
DERBIL Khaoula
HADJ SAHRAOUI Ahmed Yassir
TALBI Nazim

Soutenu devant le jury composé de :

Mr BESSAA Brahim
Mr MAYATA Raouf

N° Projet : ACAD-04

Année universitaire : 2025 - 2026

REMERCIEMENTS

Tout d'abord, nous remercions **Allah** le Tout-Puissant de nous avoir accordé la force, la persévérance et l'opportunité de mener à bien ce travail.

Nous tenons également à nous remercier **nous-mêmes** pour nos efforts, notre engagement et notre persévérance tout au long de la réalisation de ce projet.

Nous tenons à exprimer notre gratitude et remerciements à notre encadrante, Madame le Professeur Mahani Aouatef

Nos remerciements sincères vont également aux membres du **jury** qui ont accepté d'évaluer notre travail. Leur temps, leurs remarques et leurs retours constructifs nous sont infiniment précieux.

Nous remercions chaleureusement tous nos enseignants pour la qualité de leur enseignement, leurs efforts et leurs contributions à notre formation académique.

Enfin, nous adressons nos plus chaleureux remerciements à nos familles, à nos amis ainsi qu'à toutes les personnes qui, de près ou de loin, ont contribué à la réalisation de ce projet de fin d'études.

ABSTRACT

Facing the data fragmentation of the current healthcare system, this thesis presents **Healy**, an e-Health platform unifying the care pathway. It interconnects patients, practitioners, pharmacists, and home care providers through a centralized medical record, QR Code-secured prescriptions, and integrated pharmaceutical and home care management.

The major innovation lies in a pre-diagnostic assistant powered by a **RAG** (Retrieval-Augmented Generation) architecture designed to eliminate clinical hallucinations. The pipeline integrates emergency classification, *Multi-Query Expansion*, and vector encoding (*multilingual-e5-large*) for semantic search within ChromaDB. Extracted pathologies are evaluated by a *Smart Scoring* algorithm (combining cosine similarity and Jaccard index), then validated in real-time via the official **NIH** API (ICD-10 codes). This secured context is finally transmitted to the multimodal Google Gemini model to generate a reliable synthesis.

Technically, the application relies on a robust client-server architecture (React, Django, PostgreSQL). AI performance is optimized by a smart cryptographic cache (SHA-256) and a genetic algorithm autonomously calibrating the model's hyperparameters. Healy thus demonstrates the feasibility of a highly secure and interconnected Medicine 4.0.

RÉSUMÉ

Face à la fragmentation des données du système de santé actuel, ce mémoire présente **Healy**, une plateforme e-Santé unifiant le parcours de soins. Elle interconnecte patients, praticiens, pharmaciens et gardes-malades via un dossier médical centralisé, des ordonnances dématérialisées par QR Code et la gestion intégrée des prestations pharmaceutiques et à domicile.

L'innovation majeure réside dans un assistant de pré-diagnostic propulsé par une architecture **RAG** (Retrieval-Augmented Generation) conçue pour éliminer le risque d'hallucination clinique. Le pipeline intègre une classification d'urgence, un mécanisme de *Multi-Query Expansion*, et un encodage vectoriel (*multilingual-e5-large*) pour une recherche sémantique sous ChromaDB. Les pathologies extraites sont soumises à un algorithme de *Smart Scoring* (combinant similarité cosinus et indice de Jaccard), puis validées en temps réel via l'API officielle du **NIH** (codes ICD-10). Ce contexte sécurisé est enfin transmis au modèle multimodal Google Gemini pour générer une synthèse textuelle fiable.

Techniquement, l'application repose sur une architecture client-serveur robuste (React, Django, PostgreSQL). Les performances de l'IA sont optimisées par un cache intelligent cryptographique (SHA-256) et un algorithme génétique calibrant de manière autonome les hyperparamètres du modèle. Healy démontre ainsi la faisabilité d'une médecine 4.0 interconnectée et hautement sécurisée.

Keywords: e-Health, Artificial Intelligence, RAG, ChromaDB, NIH API, Smart Scoring, Client-Server Architecture.

Table des matières

CHAPITRE I	1
ÉTUDE DE L'EXISTANT ET PROBLÉMATIQUE	1
1.1 INTRODUCTION	1
1.2 DÉFINITIONS	1
1.2.1 La digitalisation du secteur de la santé (E-Santé)	1
1.2.2 L'intelligence artificielle dans le domaine médical	1
1.2.2.1 Définition et enjeux	1
1.2.2.2 Les Grands Modèles de Langage (LLM) et le risque d'hallucination	2
1.2.2.3 L'architecture RAG (Retrieval-Augmented Generation)	2
1.3 PROBLÉMATIQUE	2
1.3.1 Fonctionnement du système de santé actuel	2
1.3.1.1 Acteurs du système de santé :	2
1.3.1.2 Processus traditionnel de prise en charge :	3
1.3.2 Étude des solutions numériques existantes	3
1.3.2.1 Solutions internationales :	3
1.3.2.2 Solutions algériennes :	3
1.3.3 Synthèse et Limites de l'existant	3
1.4 OBJECTIFS	4
1.5 SOLUTION PROPOSÉE : HEALY	4
1.6 CONCLUSION	4
CHAPITRE II	5
CONCEPTION ET MODÉLISATION	5
2.1. INTRODUCTION :	5
2.2 SPÉCIFICATION DES BESOINS :	5
2.2.1. Besoins fonctionnels	5
2.2.2. Besoins non-fonctionnels	5
2.3 Représentation fonctionnelle du système	6
2.3.1 DIAGRAMMES DE CAS D'UTILISATION :	6
2.3.2 LES DIAGRAMMES DE SÉQUENCE :	9
2.3.2.1 Diagramme de séquence du scénario « Effectuer un Diagnostic IA » :	9
2.3.2.2 Diagramme de séquence du scénario « Prise de Rendez-Vous » :	9
2.3.2.3 Diagramme de séquence du scénario « Établir Ordonnance » :	11
2.3.2.4 Diagramme de séquence du scénario « Traiter offre Garde-Malade » :	12
2.3.3 DIAGRAMMES DE CLASSES :	12

2.4 SCHÉMA RELATIONNEL DE LA BASE DE DONNÉES :	14
2.5 CONCEPTION DU SYSTÈME DE DIAGNOSTIQUE PAR INTELLIGENCE ARTIFICIELLE :	15
2.5.1. Pré-traitement et stratégie Multi-Query :	15
2.5.2. Encodage et recherche vectorielle (Retrieval) :	15
2.5.3. Hybridation par Scoring Composite et Validation Clinique :	16
2.5.4. Génération Sécurisée et Optimisation Génétique :	16
2.6 CONCLUSION.....	15
CHAPITRE III.....	18
RÉALISATION ET IMPLÉMENTATION.....	18
3.1. INTRODUCTION :	18
3.2. OUTILS ET LANGAGES DE DÉVELOPPEMENT :	18
3.3. ARCHITECTURE LOGICIELLE DE LA PLATEFORME :	18
3.4. IMPLÉMENTATION DU MODULE D'INTELLIGENCE ARTIFICIELLE (IA) :	19
3.4.1. Architecture RAG et Traitement Multimodal (Gemini)	19
3.4.2. Sécurité et Validation Clinique.....	20
3.4.3. Orientation Médicale Géolocalisée.....	20
3.4.4. Optimisation des Performances et Apprentissage.....	20
3.5. PRÉSENTATION DES INTERFACES DE LA PLATEFORME :	21
3.5.1 Tableau de bord principal de l'espace patient :	21
3.5.2 Le Profil Médical Centralisé :	21
3.5.3 Le Module de Prise de Rendez-vous :	22
3.5.4 Le Tableau de bord du Médecin :	23
3.5.5 Le Module d'Ordonnance Numérique :	24
3.5.6 Page du Diagnostic Médicale IA :	24
3.6. ÉVALUATION DES PERFORMANCES ET VALIDATION DE L'ARCHITECTURE.....	25
3.6.1 Optimisation des temps de réponse (Impact du Cache SHA-256) :	25
3.6.2 Optimisation des hyperparamètres par algorithme génétique :	26
3.6.3 Validation clinique du pipeline (Impact de l'API NIH) :	27
3.6.4 Étude comparative globale : Architecture Healy (RAG) vs LLM classique.....	27
4. CONCLUSION GÉNÉRALE.....	28

Liste des figures

Figure 1 - Diagramme de cas d'utilisation : User et Patient	6
Figure 2 - Diagramme de cas d'utilisation : Médecin.....	7
Figure 3 - Diagramme de cas d'utilisation : Pharmacien.....	7
Figure 4 - Diagramme de cas d'utilisation : Garde-Malade.....	8
Figure 5 - Diagramme de cas d'utilisation : Admin.....	8
Figure 6 - Diagramme de séquence du scénario « Effectuer un Diagnostic IA ».....	9
Figure 7 - Diagramme de séquence du scénario « Prendre un rendez-vous ».....	10
Figure 8 - Diagramme de séquence du scénario « Établir Ordonnance ».....	11
Figure 9 - Diagramme de séquence du scénario « Traiter offre Garde-Malade ».....	12
Figure 10 - Diagramme de classes global.....	13
Figure 11- Flux d'exécution du pipeline RAG pour le diagnostic médical.....	17
Figure 12 - Interface du Dashboard Patient	21
Figure 13 - Interface du Profil Médical du Patient	22
Figure 14 - Interface Prise de Rendez-vous Patient	23
Figure 15 - Interface Dashboard Médecin	23
Figure 16 - Interface Création et Gestion d'Ordonnance Médecin	24
Figure 17 - Interface Diagnostic IA du Patient	25
Figure 18 - Diagramme en barre horizontale de l'Impact du Cache Intelligent sur les performances.....	25
Figure 19 - Convergence des hyperparamètres et évolution du score de fitness via l'algorithme génétique	26
Figure 20 - Impact de la validation externe (API NIH) sur la sécurisation des pré-diagnostics.....	27
Figure 21 - Analyse comparative globale (Architecture RAG vs LLM standard)	28

Liste des Abréviations

- **API** : Application Programming Interface (Interface de programmation d'application)
- **CHIFA** : Carte d'assurance maladie (Algérie)
- **CNAS** : Caisse Nationale des Assurances Sociales
- **HTTP** : Hypertext Transfer Protocol
- **IA** : Intelligence Artificielle
- **ICD-10** : International Classification of Diseases, 10th Revision (Classification internationale des maladies, 10e révision de l'OMS)
- **IDE** : Integrated Development Environment (Environnement de développement intégré)

- **JSON** : JavaScript Object Notation
- **JWT** : JSON Web Token
- **LLM** : Large Language Model (Grand Modèle de Langage)
- **MVT** : Modèle-Vue-Template
- **NIH** : National Institutes of Health
- **ORM** : Object-Relational Mapping
- **OTP** : One-Time Password (Mot de passe à usage unique)
- **PBKDF2** : Password-Based Key Derivation Function 2
- **QR Code** : Quick Response Code
- **RAG** : Retrieval-Augmented Generation (Génération Augmentée par Récupération)
- **REST** : Representational State Transfer
- **SGBD** : Système de Gestion de Base de Données
- **SGBDR** : Système de Gestion de Base de Données Relationnelle
- **SHA-256** : Secure Hash Algorithm 256-bit
- **SQL** : Structured Query Language
- **SSE** : Server-Sent Events
- **TIC** : Technologies de l'Information et de la Communication
- **UML** : Unified Modeling Language

INTRODUCTION GÉNÉRALE

Le secteur de la santé traverse une phase de mutation sans précédent, portée par l'essor de la médecine 4.0 et de l'e-santé. L'intégration de l'intelligence artificielle (IA) ouvre de nouvelles perspectives pour un suivi personnalisé et continu des patients. Cependant, le système traditionnel reste pénalisé par des processus fragmentés, manuels et fortement dépendants du support papier, ce qui provoque une rupture dans la chaîne d'information entre les différents acteurs médicaux. De plus, l'usage des Grands Modèles de Langage (LLM) en médecine se heurte à une limite critique : le risque d'hallucination clinique, incompatible avec la sécurité des soins.

C'est dans ce contexte que s'inscrit ce projet de fin d'études à travers le développement de la plateforme **Healy**. L'objectif est d'unifier le parcours de soins en connectant les patients, les médecins, les pharmaciens et les prestataires de soins à domicile au sein d'un écosystème numérique unique (dossier connecté, ordonnances QR Code, gestion pharmaceutique). L'innovation majeure réside dans un moteur de pré-diagnostic basé sur une architecture **RAG (Retrieval-Augmented Generation)**. En combinant un modèle de langage, une recherche sémantique vectorielle (ChromaDB) et une validation en temps réel via l'API officielle du **NIH**, Healy sécurise les conseils médicaux et élimine les hallucinations.

Pour présenter notre démarche, ce mémoire est structuré comme suit :

- **Le Chapitre I (Étude de l'existant et problématique)** dresse un état des lieux des pratiques actuelles et pose le cadre théorique pour justifier les objectifs du projet.
- **Le Chapitre II (Conception et modélisation)** spécifie les besoins et formalise l'architecture statique et dynamique du système à travers des diagrammes UML et la logique mathématique du pipeline.
- **Le Chapitre III (Réalisation et implémentation)** décrit l'architecture logicielle globale de la plateforme, détaille l'intégration des différents modules, présente les interfaces utilisateurs développées et expose la phase d'évaluation technique et clinique menée pour valider les performances du système.
- **Les Annexes (A, B et C)** complètent l'ouvrage en fournissant un glossaire technique, des diagrammes UML secondaires et les modèles de calcul détaillés.

CHAPITRE I

ÉTUDE DE L'EXISTANT ET PROBLÉMATIQUE

1.1 Introduction

Le secteur de la santé traverse une phase de mutation sans précédent, portée par une accélération technologique mondiale. Pourtant, malgré ces avancées, le système de santé reste marqué par des processus archaïques, souvent manuels et fragmentés, qui peinent à répondre efficacement à la multiplication des acteurs et à la complexité des parcours de soins. Face à ces défis, l'émergence d'une plateforme capable de centraliser les flux d'informations de manière intelligente devient une nécessité impérieuse. Ce chapitre dresse un état des lieux exhaustif du fonctionnement traditionnel de la santé, explore les fondements de la révolution numérique et de l'intelligence artificielle, et analyse les solutions actuelles afin de justifier le développement de notre projet: **Healy**.

1.2 Définitions

Cette section pose les bases théoriques indispensables pour comprendre les enjeux de la modernisation du secteur médical.

1.2.1 La digitalisation du secteur de la santé (E-Santé)

La digitalisation, ou "e-santé", désigne l'intégration des technologies de l'information et de la communication (TIC) au service de la santé. Elle ne se limite pas à la simple numérisation de documents, mais englobe la dématérialisation complète des processus : ordonnances électroniques sécurisées, téléconsultations, partage en temps réel des dossiers médicaux entre spécialistes et automatisation des circuits administratifs. Bien que cette transformation promette une efficacité accrue, son adoption reste hétérogène, se heurtant souvent à des barrières d'infrastructures ou de culture numérique.

1.2.2 L'intelligence artificielle dans le domaine médical

L'IA s'est imposée comme le moteur de la « médecine 4.0 », permettant de passer d'une approche réactive à une médecine plus prédictive et personnalisée.

1.2.2.1 Définition et enjeux

L'intelligence artificielle (IA) vise à simuler des processus cognitifs humains — tels que l'apprentissage et le raisonnement — par des algorithmes complexes. Dans le secteur de la santé, son intégration offre des bénéfices concrets :

- **Optimisation du temps médical** : Réalisation de pré-diagnostic pour libérer du temps de consultation.
- **Sécurisation du parcours** : Réduction des risques d'erreurs humaines par l'analyse de données massives.
- **Accessibilité** : Offre au patient une assistance et un suivi 24h/24, comblant le vide entre deux rendez-vous.

1.2.2.2 Les Grands Modèles de Langage (LLM) et le risque d'hallucination

Au cœur de l'intelligence artificielle textuelle se trouvent les **LLM** (*Large Language Models*, ou Grands Modèles de Langage). Ce sont des algorithmes entraînés sur des milliards de textes pour comprendre et générer du langage humain.

Cependant, malgré leur puissance, ces modèles présentent une limite structurelle majeure en médecine : l'**hallucination**. Étant basés sur des probabilités statistiques plutôt que sur un raisonnement médical, ils peuvent générer des réponses syntaxiquement parfaites mais factuellement fausses (inventions de maladies ou de traitements). De plus, leur connaissance est « figée » à leur date d'entraînement, ce qui les rend aveugles aux dernières données cliniques ou aux antécédents spécifiques d'un patient.

1.2.2.3 L'architecture RAG (Retrieval-Augmented Generation)

Pour pallier ces risques et garantir une sécurité maximale, le paradigme de la **Génération Augmentée par Récupération (RAG)** s'est imposé. Contrairement à une IA classique, le RAG force le système à consulter des sources externes vérifiées avant de formuler une réponse.

Ce processus se décompose en trois étapes :

1. **La Récupération (Retrieval)** : Le système cherche dans une base de connaissances médicale structurée les informations pertinentes liées aux symptômes.
2. **L'Augmentation** : Ces données fiables et à jour sont injectées dans le contexte de l'IA.
3. **La Génération sécurisée** : L'assistant rédige sa réponse en se basant exclusivement sur ce contexte validé, éliminant ainsi les divagations et garantissant la traçabilité des sources.

1.3 Problématique

Pour concevoir une solution pertinente, il est nécessaire de comprendre les failles du modèle actuel.

1.3.1 Fonctionnement du système de santé actuel

1.3.1.1 Acteurs du système de santé : Le système repose sur un réseau d'acteurs dont les interactions sont constantes :

- **Patient** : au cœur du système, il consulte, achète des médicaments et se fait rembourser.
- **Médecin** : professionnel de santé qui diagnostique et prescrit des traitements.
- **Pharmacien** : délivre les médicaments sur ordonnance et conseille les patients.
- **Assurance (CNAS / CHIFA)** : organisme de prise en charge et de remboursement des frais médicaux.
- **Laboratoires** : effectuent des analyses biologiques et médicales.

1.3.1.2 Processus traditionnel de prise en charge : Le parcours actuel est majoritairement physique et dépendant du support papier. Une prise en charge classique oblige le patient à multiplier les déplacements et les documents : consultation physique, obtention d'une ordonnance manuscrite, passage en pharmacie, et gestion manuelle des justificatifs pour les remboursements.

1.3.2 Étude des solutions numériques existantes

1.3.2.1 Solutions internationales : Des plateformes comme *Doctolib* ont révolutionné la prise de rendez-vous, tandis qu'*Ada Health* utilise l'IA pour l'évaluation symptomatique. Cependant, ces outils sont souvent "en silos", forçant l'utilisateur à jongler entre plusieurs applications pour gérer l'ensemble de son parcours médical.

1.3.2.2 Solutions algériennes : En Algérie, le système *CHIFA* a marqué une étape clé dans la gestion des remboursements. Plus récemment, la transition numérique s'est accélérée avec l'apparition de plateformes dédiées à la santé de proximité comme *Sihati24.dz*. Cette solution algérienne permet de localiser instantanément les pharmacies de garde, les pharmacies classiques et les parapharmacies. Elle intègre également un service pour réserver des soins à domicile avec des professionnels qualifiés tels que des médecins, des infirmiers ou des kinésithérapeutes. Bien que cette initiative soit très utile pour connecter les patients aux praticiens, elle reste principalement focalisée sur la mise en relation et manque d'outils médicaux avancés.

1.3.3 Synthèse et Limites de l'existant

L'analyse de l'existant révèle des lacunes structurelles majeures :

- **Vraie rupture de la chaîne d'information** : Même avec des plateformes locales comme *Sihati24*, l'ordonnance reste souvent au format papier, s'égare facilement, et il n'y a pas de traçabilité numérique sécurisée entre le médecin prescripteur et le pharmacien.
- **Fragmentation des données** : L'historique médical d'un patient n'est jamais centralisé, obligeant chaque nouveau médecin à repartir de zéro, ce qui nuit à la qualité des soins.

- **Absence d'assistance IA au diagnostic** : Les annuaires actuels aident à trouver un professionnel, mais ils oublient une étape cruciale : aider le patient à comprendre ses symptômes et à évaluer le degré d'urgence *avant* la consultation.

1.4 Objectifs

Face à ce constat, notre projet se fixe pour missions de :

- **Unifier le parcours patient** en regroupant tous les intervenants sur une plateforme unique.
- **Sécuriser le circuit du médicament** via la dématérialisation et l'usage de QR Codes.
- **Humaniser l'IA** pour en faire un véritable assistant capable de comprendre, de rassurer et d'orienter le patient.
- **Faciliter le maintien à domicile** en professionnalisant l'accès aux services de gardes-malades.

1.5 Solution Proposée : Healy

Pour répondre à ces besoins, nous avons développé **Healy**, une plateforme globale qui se distingue par :

- **Un moteur de diagnostic par IA** : Capable d'évaluer les symptômes, de lire des fichiers médicaux (PDF/Images) et de déterminer le degré d'urgence.
- **Un écosystème pharmaceutique** : Ordonnances numériques, gestion des stocks en pharmacie et commandes en ligne sécurisées.
- **Un dossier médical intelligent** : Centralisation des antécédents et interprétation vulgarisée des analyses pour une meilleure compréhension par le patient.
- **Un module de soins à domicile** : Mise en relation directe avec des gardes-malades qualifiés, basée sur le profil spécifique du patient.
- **Une sécurité renforcée** : Vérification stricte des diplômes et identités des professionnels, géolocalisation pour les cas d'urgence et consultation des informations d'assurance.

1.6 Conclusion

L'étude de l'existant démontre que si le système de santé algérien s'est engagé sur la voie du numérique, il reste encore de nombreux "vides technologiques" à combler. Les solutions actuelles, bien qu'utiles, restent trop parcellaires. C'est précisément pour offrir une réponse cohérente, sécurisée et intelligente à l'ensemble des acteurs de la santé que nous avons conçu **Healy**. Les chapitres suivants détailleront la conception technique de cette solution

CHAPITRE II

CONCEPTION ET MODÉLISATION

2.1. Introduction :

Après avoir défini le contexte et les objectifs de notre projet, ce deuxième chapitre va nous permettre de passer de la théorie à la pratique. L'idée ici est de traduire les besoins que nous avons identifiés en une véritable architecture logicielle. Pour modéliser tout cela de manière claire, nous avons choisi de nous appuyer sur le langage UML (*Unified Modeling Language*). Cela va nous aider à schématiser le fonctionnement de notre plateforme sous tous ses angles avant de nous lancer dans le développement de l'application.

2.2 Spécification des besoins :

Pour que notre plateforme soit vraiment utile, il fallait d'abord définir clairement ce qu'elle devait faire, mais aussi les contraintes techniques qu'elle devait respecter.

2.2.1. Besoins fonctionnels

Le système a été pensé pour répondre aux attentes de tous nos utilisateurs à travers plusieurs fonctionnalités clés :

- **Gestion des comptes** : Permettre l'inscription, la connexion et la gestion des différents types de profils (Patient, Médecin, Pharmacien, Garde-malade, Administrateur).
- **Dossier Médical** : Offrir au patient un espace privé pour centraliser ses informations de santé (allergies, antécédents, traitements en cours).
- **Consultations** : Faciliter la prise de rendez-vous en ligne, le suivi des visites médicales et la création d'ordonnances numériques sécurisées via un QR code.
- **Pharmacie** : Permettre aux pharmaciens de gérer facilement leur stock et aux patients de commander leurs médicaments directement depuis l'application.
- **Soins à domicile** : Intégrer un outil pour chercher et réserver un garde-malade en fonction de ses disponibilités, de son expérience et de ses tarifs.
- **Assistance IA** : Mettre à disposition un assistant virtuel intelligent capable d'analyser les symptômes du patient, d'évaluer l'urgence de la situation et de l'orienter vers le bon médecin.

2.2.2. Besoins non-fonctionnels

Au-delà des fonctionnalités visibles, l'application doit aussi respecter des exigences de qualité :

- **Sécurité des données** : Puisque nous manipulons des données médicales sensibles, il est impératif de sécuriser les mots de passe et de protéger l'accès aux dossiers des patients.
- **Performances** : L'interaction avec l'Intelligence Artificielle doit être rapide et fluide pour donner l'impression d'une vraie discussion, sans temps d'attente trop longs.
- **Évolutivité** : Le code et la base de données doivent être pensés pour pouvoir accueillir de futurs utilisateurs ou de nouvelles fonctionnalités sans avoir à tout recoder.

2.3 Représentation fonctionnelle du système

Pour modéliser les interactions entre les acteurs et notre plateforme, nous avons choisi le langage UML (*Unified Modeling Language*). Ce standard universel orienté objet nous permet de structurer la conception de notre système selon trois axes complémentaires : l'aspect fonctionnel via les diagrammes de cas d'utilisation, l'aspect statique avec le diagramme de classes, et l'aspect dynamique à travers les diagrammes de séquence.

2.3.1 Diagrammes de cas d'utilisation :

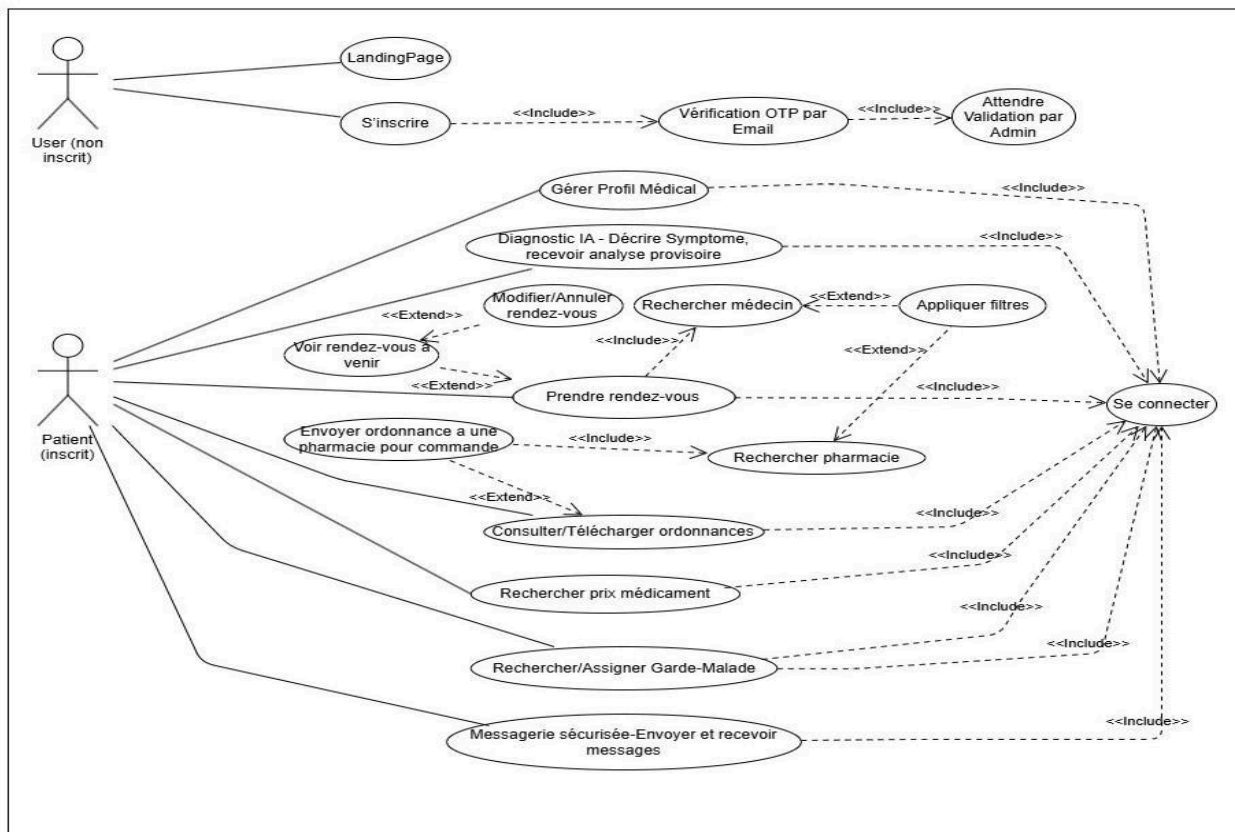


Figure 1 - Diagramme de cas d'utilisation : User et Patient

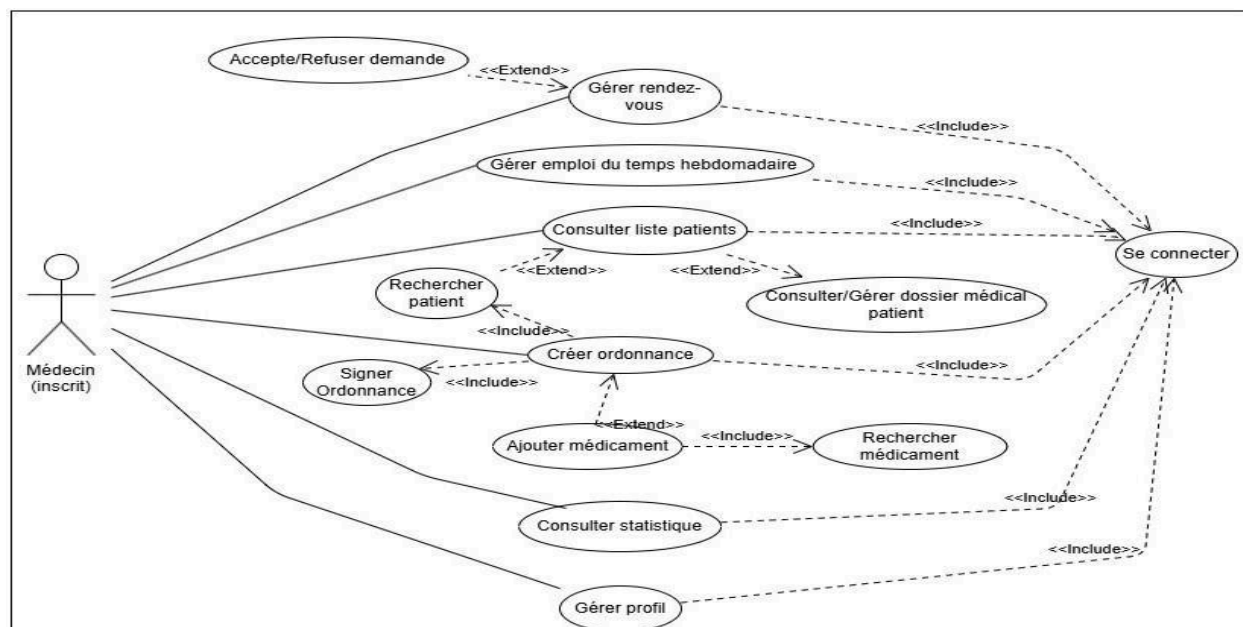


Figure 2 - Diagramme de cas d'utilisation : Médecin

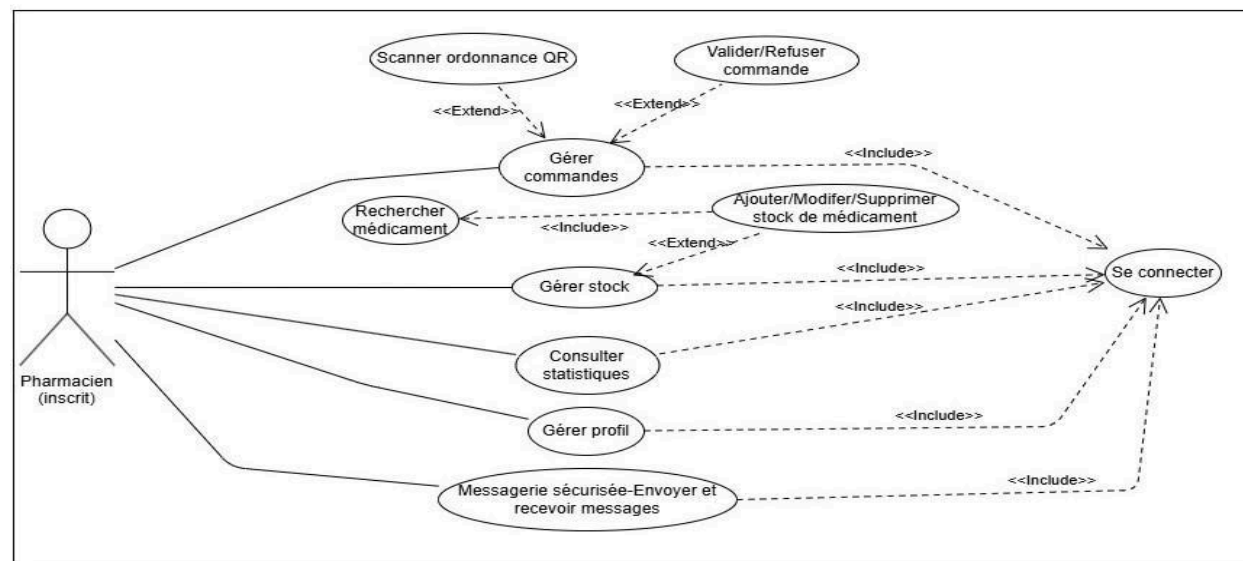


Figure 3 - Diagramme de cas d'utilisation : Pharmacien

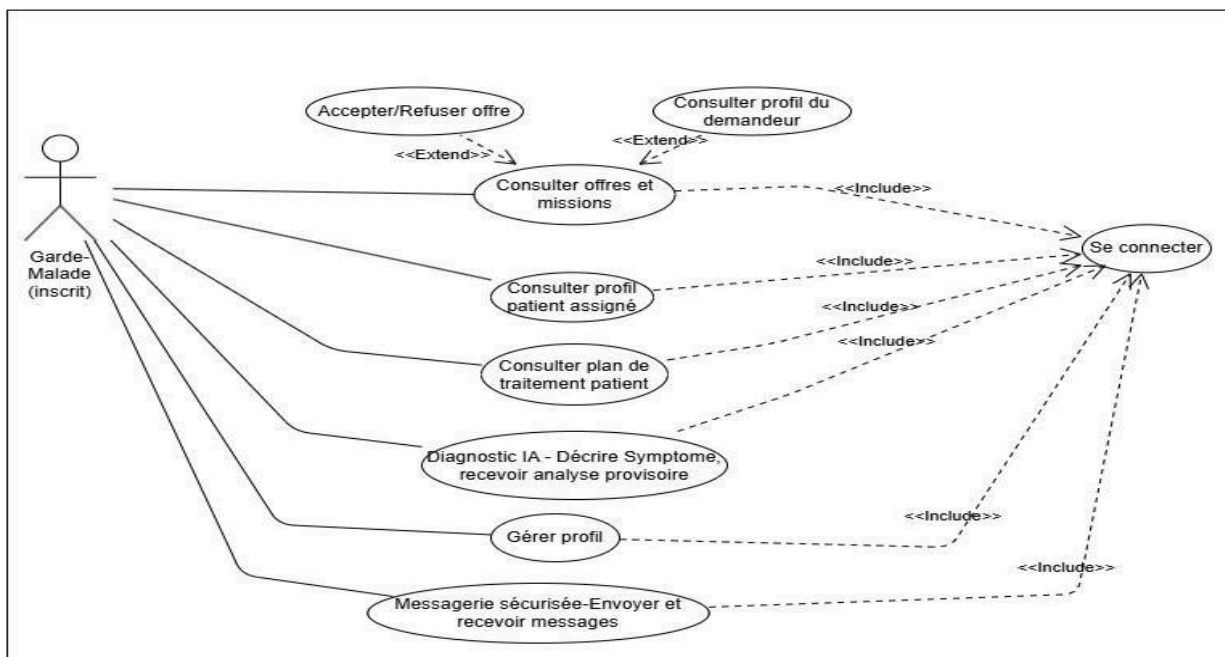


Figure 4 - Diagramme de cas d'utilisation : Garde-Malade

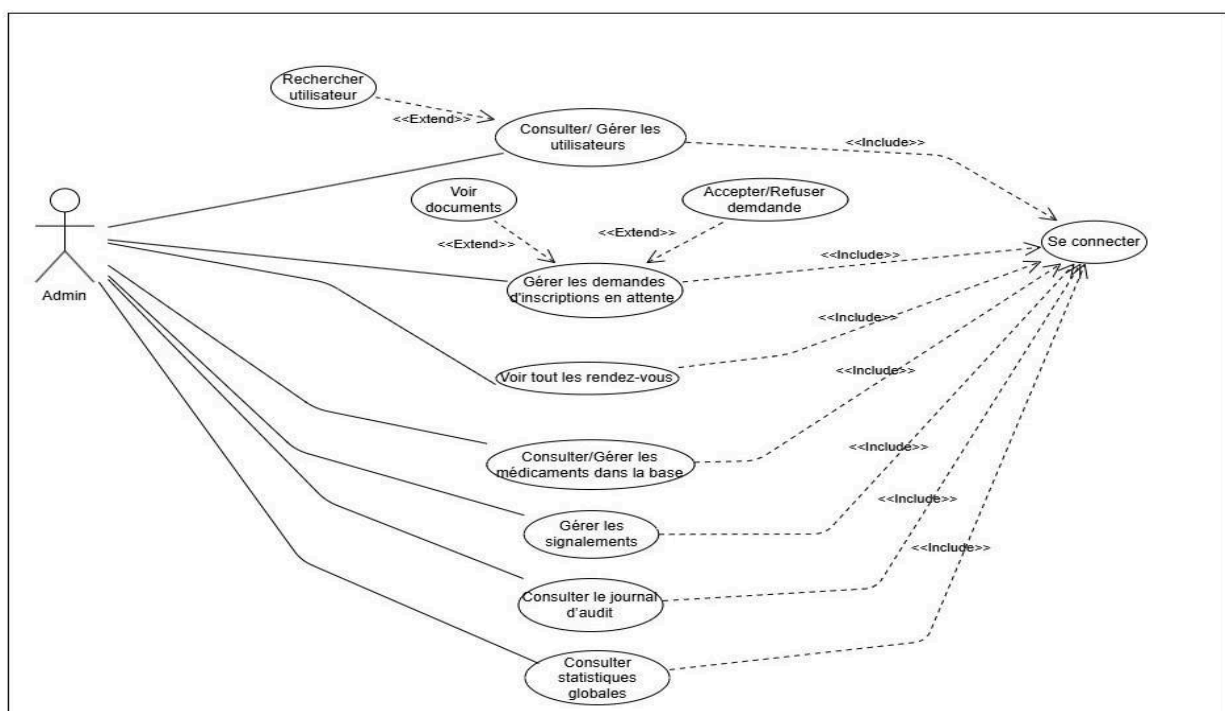


Figure 5 - Diagramme de cas d'utilisation : Admin

2.3.2 Les diagrammes de séquence :

2.3.2.1 Diagramme de séquence du scénario « Effectuer un Diagnostic IA » :

Ce diagramme modélise le flux d'exécution de l'assistant de pré-diagnostic intelligent. Il met en évidence l'architecture RAG (Retrieval-Augmented Generation) côté backend : après la saisie des symptômes, le système récupère le contexte médical du patient, effectue une recherche sémantique vectorielle (ChromaDB), procède à une validation clinique, puis génère une réponse personnalisée via le modèle Gemini avant de la restituer sur l'interface utilisateur.

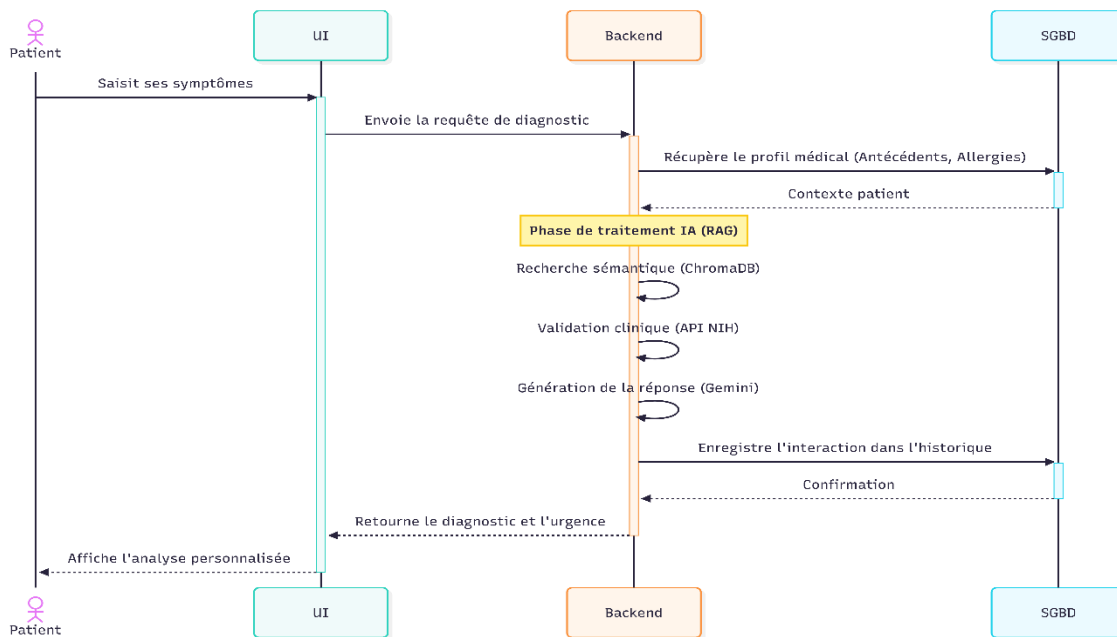


Figure 6 - Diagramme de séquence du scénario « Effectuer un Diagnostic IA »

2.3.2.2 Diagramme de séquence du scénario « Prise de Rendez-Vous » :

Ce diagramme illustre le processus complet de réservation d'une consultation médicale. Il détaille les interactions entre le patient, qui recherche un praticien selon des critères précis et sélectionne un créneau disponible, et le médecin, qui est notifié pour valider ou décliner la demande. Le système gère de manière autonome les mises à jour d'état dans la base de données (SGBD) et le routage des notifications asynchrones.

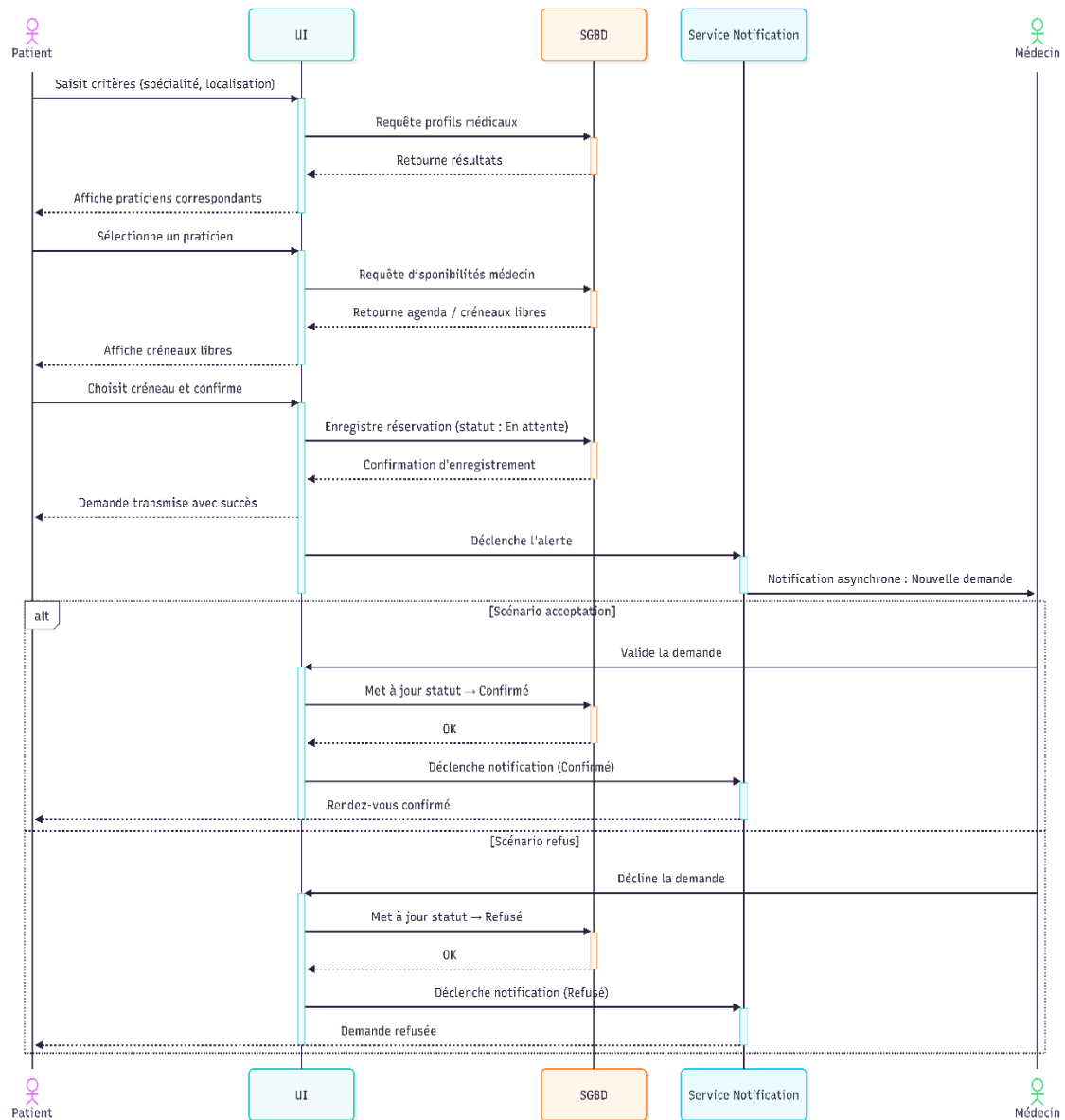


Figure 7 - Diagramme de séquence du scénario « Prendre un rendez-vous »

2.3.2.3 Diagramme de séquence du scénario « Établir Ordonnance » :

Ce diagramme retrace les étapes de création d'une ordonnance médicale électronique sécurisée. Il montre l'interaction du médecin sélectionnant un patient et prescrivant la posologie. Le traitement backend inclut la sécurisation du document par la génération d'une signature numérique et d'un code QR garantissant l'authenticité de la prescription. Une fois l'enregistrement effectué, un flux asynchrone alerte immédiatement le patient de la disponibilité de son ordonnance.

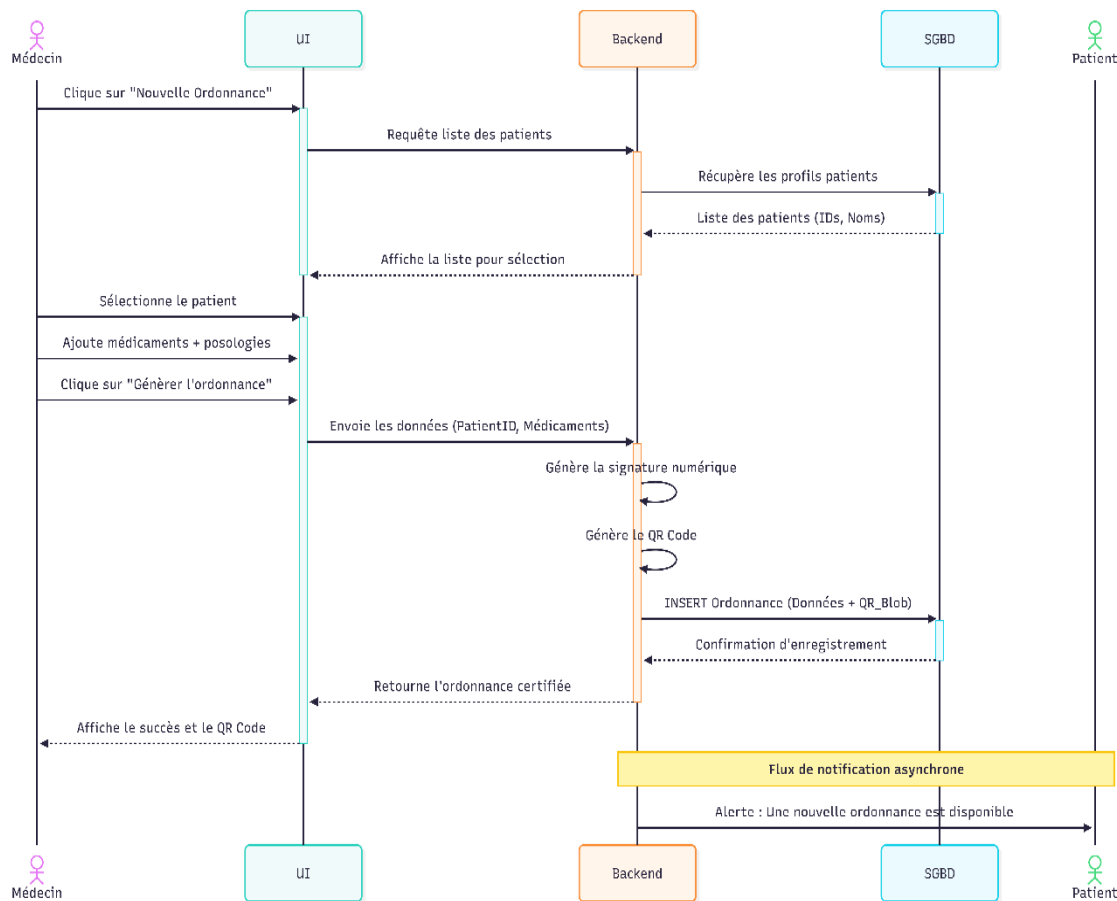


Figure 8 - Diagramme de séquence du scénario « Établir Ordonnance »

2.3.2.4 Diagramme de séquence du scénario « Traiter offre Garde-Malade » :

Ce schéma décrit le processus de mise en relation pour les services de soins à domicile. Il se divise en trois phases clés : la consultation des missions ouvertes par le garde-malade, la soumission de sa candidature pour une offre spécifique, et enfin le traitement de cette demande par le patient concerné. Ce dernier évalue le profil du postulant et émet une décision (acceptation ou refus), déclenchant ainsi l'affectation finale et les notifications.

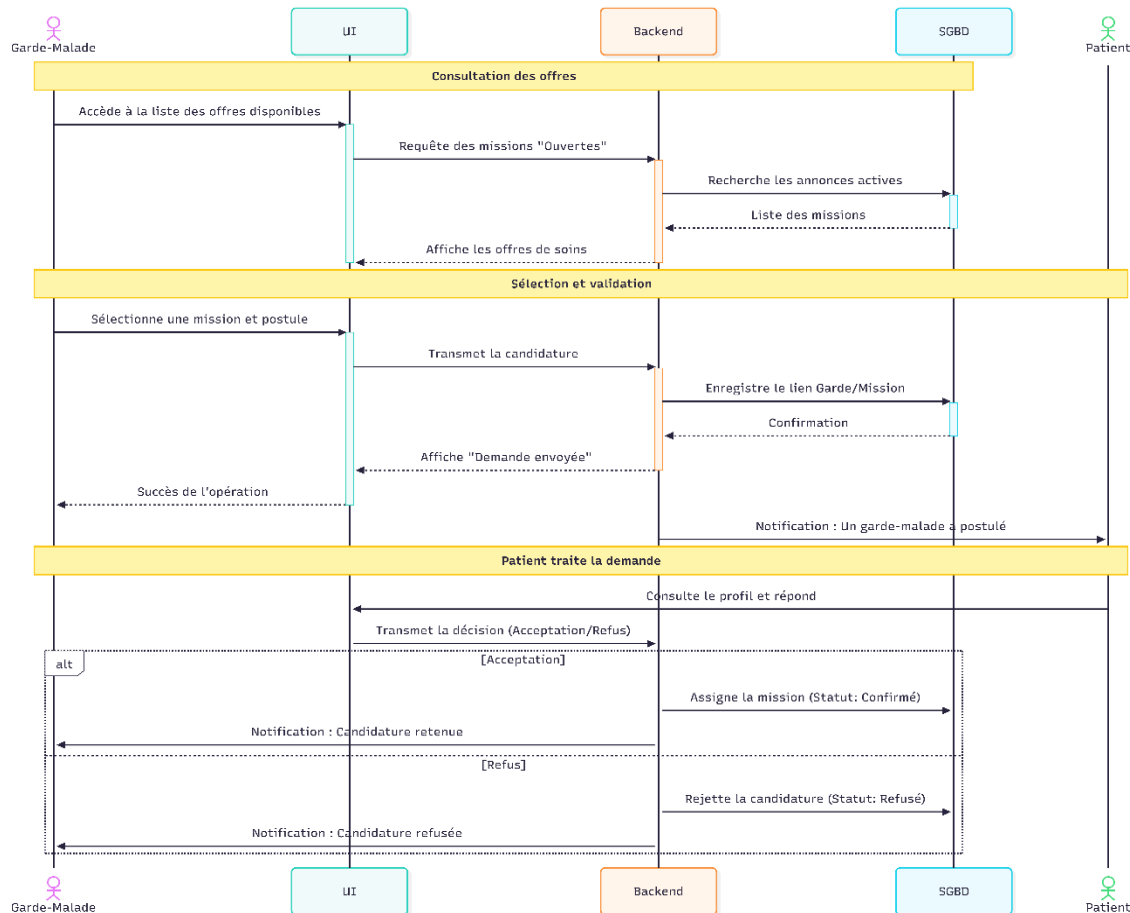


Figure 9 - Diagramme de séquence du scénario « Traiter offre Garde-Malade »

2.3.3 Diagrammes de classes :

Le diagramme de classes global modélise l'architecture statique de la plateforme Healy, structurée autour des profils utilisateurs (Patient, Médecin, Pharmacien, Garde-Malade) héritant d'une entité *User* centrale. Il illustre les relations complexes entre la gestion des rendez-vous, le dossier médical, le système de prescriptions sécurisées par QR code et le module d'intelligence artificielle. Afin d'alléger la lecture de ce chapitre, une vue globale est présentée ci-dessous. Les diagrammes détaillés, isolant chaque sous-système pour une meilleure lisibilité, sont disponibles dans l'Annexe B.

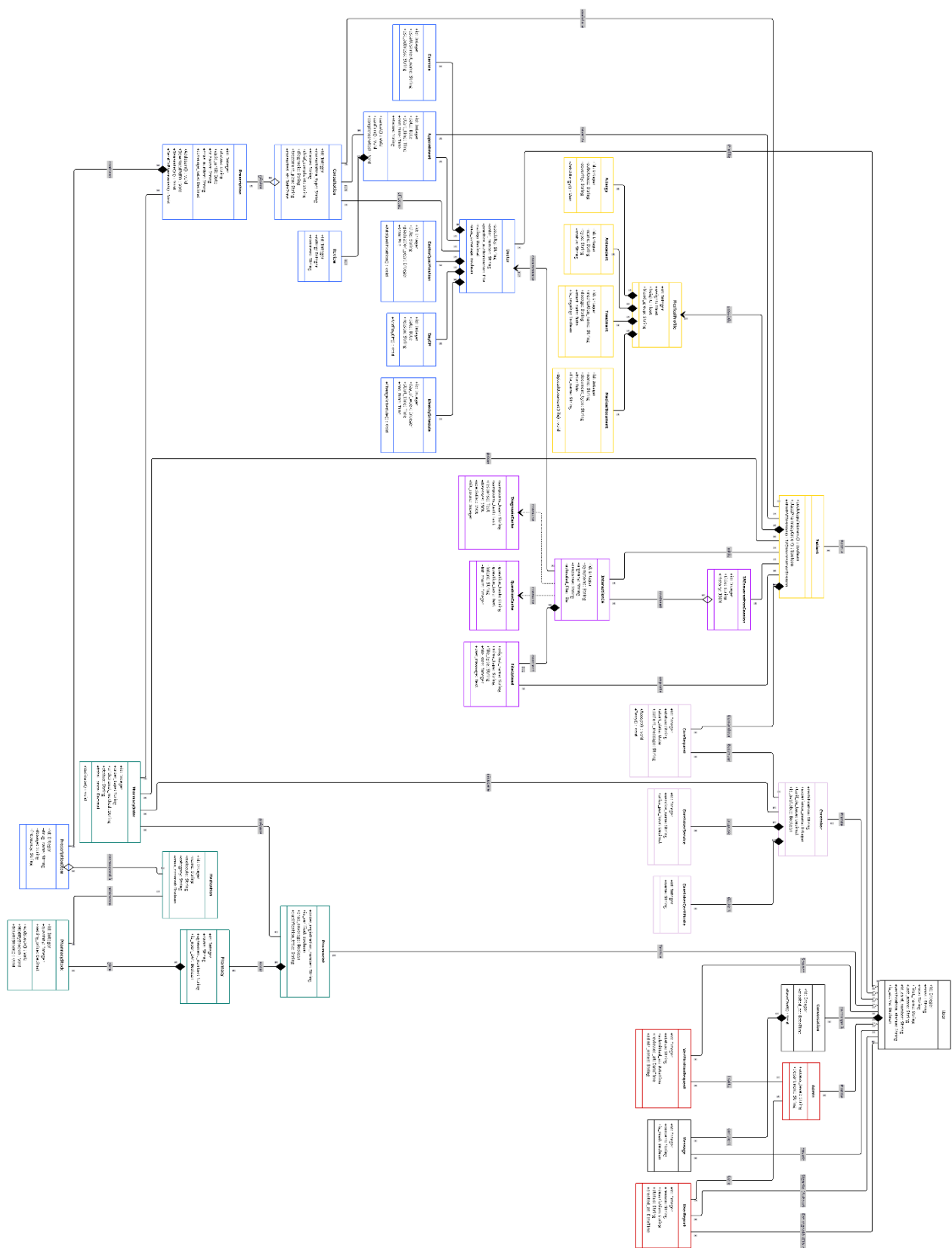


Figure 10 - Diagramme de classes global

2.4 Schéma Relationnel de la Base de Données :

- **USER** (id, email, first_name, last_name, role, phone, city, wilaya, verification_status)
- **PATIENT** (id, #user_id)
- **DOCTOR** (id, #user_id, specialty, order_number, is_verified, consultation_fee, rating)
- **PHARMACIST** (id, #user_id, order_registration_number, is_verified)
- **CARETAKER** (id, #user_id, certification, experience_years, is_verified, is_available, tarif_de_base)
- **MEDICAL_PROFILE** (id, #patient_id, weight, height, blood_group, emergency_contact_name)
- **ALLERGY** (id, #medical_profile_id, substance, severity)
- **ANTECEDENT** (id, #patient_id, name, type, status, date_diagnosis)
- **TREATMENT** (id, #patient_id, #doctor_id, #medication_id, dosage, frequency, start_date, end_date)
- **MEDICAL_DOCUMENT** (id, #patient_id, name, document_type, date)
- **Patients_SymptomAnalysis** (id, #patient_id, symptoms, possible_diagnosis, severity_level, date)
- **AI_SESSION** (id, #patient_id, title, history, lang, is_active)
- **AI_INTERACTION** (id, #patient_id, #recommended_doctor_id, symptoms, response, urgency, lang)
- **APPOINTMENT** (id, #patient_id, #doctor_id, date, start_time, end_time, motif, status)
- **CONSULTATION** (id, #doctor_id, #patient_id, #appointment_id, type, status, chief_complaint, diagnosis, consulted_at)
- **REVIEW** (id, #appointment_id, #patient_id, #doctor_id, rating, comment)
- **MEDICATION** (id, name, molecule, category, form, requires_prescription, price_dzd)
- **PRESCRIPTION** (id, #consultation_id, status, valid_until)
- **PRESCRIPTION_ITEM** (id, #prescription_id, #medication_id, drug_name, dosage, frequency, duration, quantity)
- **QR_TOKEN** (id, #prescription_id, token, is_used, expires_at)
- **PHARMACY** (id, #pharmacist_id, name, pharm_city, latitude, longitude, is_open_24h)
- **PHARMACY_STOCK** (id, #pharmacy_id, #medication_id, quantity, selling_price, expiry_date)
- **PHARMACY_ORDER** (id, #patient_id, #prescription_id, #pharmacist_id, order_type, status, total_price)
- **CARE_REQUEST** (id, #patient_id, #caretaker_id, status, start_date, end_date, patient_message)
- **CARE_MESSAGE** (id, #care_request_id, #sender_id, content, created_at)
- **CONVERSATION** (id)
- **CONVERSATION_PARTICIPANT** (#conversation_id, #user_id)

- **MESSAGE** (id, #conversation_id, #sender_id, content, is_read)
- **NOTIFICATION** (id, #user_id, title, notification_type, is_read)

2.5 Conception du système de diagnostique par Intelligence Artificielle :

Dans notre solution, nous souhaitons concevoir un assistant médical intelligent capable d'analyser les symptômes d'un patient pour proposer des pré-diagnosics précis, tout en s'adaptant à la langue et à la clarté de la requête. L'objectif est d'offrir une orientation fiable tout en garantissant une sécurité clinique absolue afin d'éviter les "hallucinations" fréquentes des modèles de langage standards. Pour cela, nous allons appliquer une architecture hybride RAG (Retrieval-Augmented Generation) mêlant recherche sémantique multilingue, règles cliniques et génération contrôlée.

2.5.1. Pré-traitement et stratégie Multi-Query : Pour préparer l'entrée envoyée à notre base de données vectorielle, nous construisons une requête enrichie en suivant une approche progressive. D'abord, nous appliquons une cascade de filtres pour détecter l'intention (rejet des questions non médicales et accumulation de l'historique des symptômes). Ensuite, nous utilisons une stratégie "Multi-Query". Les symptômes saisis (en français ou en arabe) sont traduits en anglais grâce à un dictionnaire médical local, complété par un traducteur externe si nécessaire. Le système génère plusieurs variantes de la requête (intégration de synonymes et termes cliniques). Notre objectif est d'envoyer les données les plus exhaustives possibles pour pallier les imprécisions de vocabulaire des patients et maximiser les chances de trouver la bonne pathologie.

2.5.2. Encodage et recherche vectorielle (Retrieval) : Dans notre système, nous souhaitons intégrer une approche basée sur le contenu sémantique en nous appuyant sur le modèle *multilingual-e5-large*. Ce modèle a pour rôle de transformer les caractéristiques textuelles (symptômes du patient et fiches maladies) en représentations vectorielles denses de 1024 dimensions. Pour des raisons strictes de confidentialité des données de santé et d'optimisation de la latence, nous avons fait le choix d'exécuter ce modèle entièrement en mode hors-ligne.

Ces embeddings sont sauvegardés dans une base de données ChromaDB. Pour gérer intelligemment les résultats, nous avons sciemment séparé nos données en deux collections : *medical_common* (maladies courantes) et *medical_rare* (pathologies rares). Lors d'une requête, la similarité cosinus permet de comparer le vecteur des symptômes avec ceux des maladies.

Mathématiquement, pour un vecteur de requête q (symptômes) et un vecteur documentaire d (maladie) préalablement normalisés, cette similarité se modélise ainsi :

$$\text{Similarité}(q, d) = \frac{q \cdot d}{||q|| \times ||d||}$$

Afin de rendre cette distance vectorielle exploitable par nos algorithmes de décision cliniques, la distance brute retournée par ChromaDB est convertie en un score de confiance normalisé entre 0 et 1 :

$$Confiance = \max\left(0.0, 1.0 - \frac{Distance}{2.0}\right)$$

Pour contrer le biais des IA qui sous-représentent les maladies rares, nous appliquons un mécanisme de priorité : si une maladie rare obtient un score de similarité supérieur d'au moins 0.03 au meilleur score d'une maladie commune, elle est automatiquement promue en tête des suggestions.

2.5.3. Hybridation par Scoring Composite et Validation Clinique : Pour tirer parti des différentes méthodes d'évaluation, nous allons appliquer une hybridation par agrégation des résultats. Plutôt que de nous fier uniquement à la distance mathématique, nous avons conçu un algorithme de scoring composite pour classer les maladies candidates. Le score final est calculé en fusionnant plusieurs critères :

- La similarité vectorielle (30%).
- Le taux de correspondance exact des symptômes (25%).
- Une logique médicale stricte (20%), qui applique des pénalités sévères (-1.5) si des symptômes d'exclusion sont détectés.
- Le niveau d'urgence (10%)

Afin d'ajouter une couche de validation externe (15% du score final), notre système interroge en temps réel l'API officielle de la *National Library of Medicine* (NIH). Cette étape permet de vérifier l'existence des codes ICD-10 associés aux maladies trouvées. Une maladie validée voit sa confiance augmentée, garantissant que nous ne manipulons que des données médicalement reconnues.

2.5.4. Génération Sécurisée et Optimisation Génétique : Le diagnostic final se produit lorsque les maladies les plus probables ont été identifiées et filtrées. Ces informations fiables (noms, urgence, spécialité requise) sont fusionnées avec le dossier du patient (allergies, antécédents) pour créer un contexte strict. Ce contexte est envoyé au modèle de langage (Google Gemini) qui agit uniquement comme un moteur de synthèse pour rédiger une réponse rassurante et vulgarisée.

Enfin, pour s'assurer que notre assistant reste précis et constant, nous avons intégré un algorithme génétique d'amélioration continue. Ce mécanisme évalue régulièrement la qualité des réponses de l'IA par rapport à un jeu de données de référence (cas de test). L'évaluation de chaque configuration repose sur une fonction de performance (Fitness) qui pondère rigoureusement l'exactitude clinique et la qualité conversationnelle de la réponse générée :

$$Fitness = 0.40 \times S_{disease} + 0.30 \times S_{quality} + 0.20 \times S_{disclaimer} + 0.10 \times S_{length}$$

où $S_{disease}$ représente le ratio de maladies correctement identifiées par rapport aux attentes du cas de test.

En cherchant à maximiser ce score de *Fitness*, le mécanisme ajuste ensuite automatiquement les hyperparamètres du modèle de langage (tels que la température et le *top_p*) afin de converger vers la configuration offrant les meilleurs résultats cliniques.

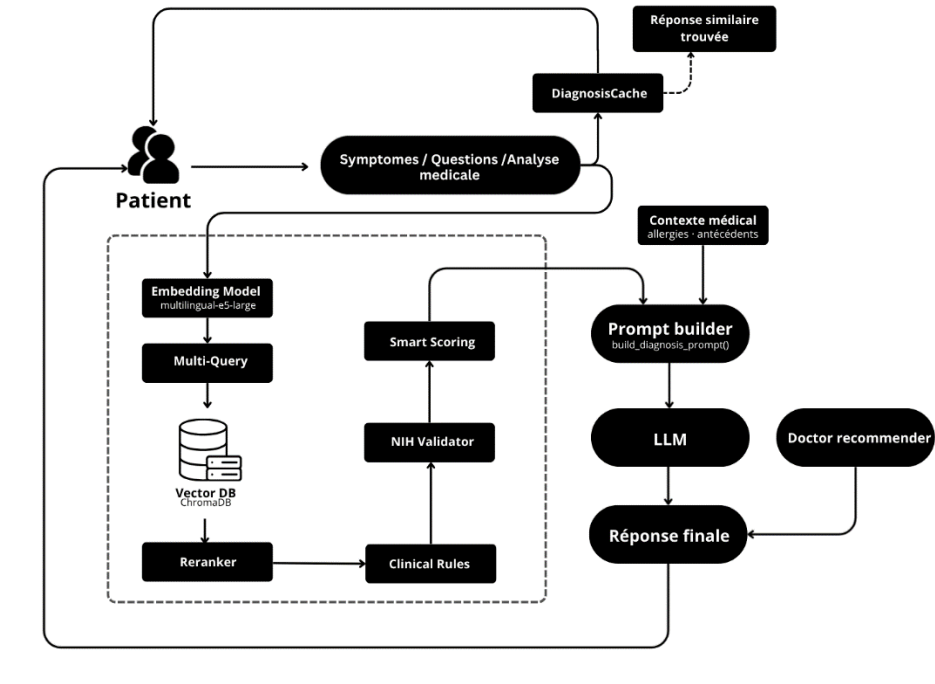


Figure 11 - Flux d'exécution du pipeline RAG pour le diagnostic médical

2.6 Conclusion

Les modèles UML et les spécifications techniques établis dans ce chapitre constituent un plan directeur solide et clair pour la phase d'implémentation. L'architecture d'intelligence artificielle basée sur le RAG offre une solution innovante au problème critique des hallucinations médicales, garantissant ainsi que la sécurité du patient demeure la priorité absolue. Le Chapitre III présentera la réalisation concrète de cette conception.

CHAPITRE III

RÉALISATION ET IMPLÉMENTATION

3.1. Introduction :

Après avoir posé les fondations théoriques de notre projet lors de la phase de conception, ce troisième chapitre nous plonge dans le concret : la réalisation technique. Nous allons y détailler les outils, les langages et les choix d'architecture qui nous ont permis de donner vie à notre plateforme. Enfin, nous ferons un tour d'horizon du résultat final en présentant les interfaces que nous avons développées et le parcours type de nos utilisateurs.

3.2. Outils et langages de développement :

Pour construire un écosystème aussi complet, nous avons dû sélectionner des technologies modernes et adaptées à nos besoins, particulièrement pour coupler la logique web classique avec notre moteur d'Intelligence Artificielle.

Visual Studio Code (VS Code) : Environnement de développement (IDE) principal, choisi pour sa légèreté, sa rapidité et son vaste écosystème d'extensions.

Python : Langage de programmation principal, choisi pour sa puissance et sa prédominance dans le domaine du traitement de données et de l'IA.

Django : Framework web Python robuste, utilisé pour structurer le Back-End et gérer efficacement les modèles de données complexes grâce à son ORM natif.

Modèles de langage (LLMs) : Intégrés au moteur IA pour analyser les symptômes des patients, animer les sessions de diagnostic et formuler des recommandations médicales.

PostgreSQL : Système de gestion de base de données relationnelle (SGBDR) robuste et sécurisé, garantissant l'intégrité et la persistance des données médicales.

Postman : Plateforme indispensable utilisée pour exécuter, tester et valider les requêtes de notre API REST (communication entre le Front-End et le Back-End).

React et Node.js : Bibliothèque JavaScript (React) couplée à son environnement d'exécution (Node.js), utilisés pour développer une interface utilisateur (Front-End) fluide, dynamique et réactive.

Git & GitHub : Outils de contrôle de version utilisés pour gérer l'historique du code source et assurer la sauvegarde continue du projet.

Visual Paradigm et Mermaid.js : Outils de modélisation avancés, exploités pour concevoir l'architecture du système et générer nos diagrammes UML.

3.3. Architecture Logicielle de la Plateforme :

L'architecture de notre application s'appuie sur un modèle client-serveur structuré autour d'une API RESTful. Les interfaces utilisateur, conçues avec React, communiquent avec le serveur

(Back-End) via des requêtes HTTP standards (GET, POST, PUT, DELETE), respectant ainsi les principes fondamentaux de l'architecture REST. Cette API fixe les règles d'échange, tandis que le format JSON assure une circulation fluide des données entre les deux parties.

Le Back-End, développé en Python avec Django, suit une architecture MVT (Modèle-Vue-Template). Dans notre approche, React prend en charge la couche de présentation, pendant que Django pilote toute la logique métier. Par exemple, lorsqu'un patient interagit avec l'application, les données transitent par une route REST avant d'être interceptées par une « Vue » Django. C'est elle qui applique les règles de gestion et sollicite les « Modèles », lesquels utilisent l'ORM natif de Django pour interagir avec la base de données PostgreSQL.

Bien entendu, comme nous manipulons des données de santé, la sécurité a été l'une de nos priorités. L'authentification repose sur un système de jetons sécurisés (Tokens JWT) générés lors de la connexion. Les mots de passe des utilisateurs sont systématiquement hachés par les algorithmes cryptographiques natifs de Django (comme PBKDF2) avant d'être stockés, et le fait de passer par l'ORM de Django pour nos requêtes SQL nous protège naturellement contre les failles d'injection.

3.4. Implémentation du Module d'Intelligence Artificielle (IA) :

Le module de diagnostic IA constitue le cœur intelligent de notre plateforme. Afin de garantir des résultats rapides, pertinents et médicalement sécurisés, son implémentation repose sur la combinaison de plusieurs méthodes et outils avancés :

3.4.1. Architecture RAG et Traitement Multimodal (Gemini)

Étude comparative et choix du Modèle de Langage (LLM) :

Lors de la conception, plusieurs Modèles de Langage (LLM) ont été évalués. Si la famille GPT (OpenAI) ou les modèles open-source (Llama) offrent d'excellentes capacités de raisonnement textuel, ils présentent certaines limites pour notre cas d'usage clinique : des fenêtres de contexte (limite de *tokens*) souvent trop restreintes pour ingérer l'intégralité d'un long historique patient, et un traitement des images rajouté *a posteriori*. Notre choix s'est donc porté sur Google Gemini. Doté d'une très large capacité de *tokens*, il se distingue surtout par son architecture nativement multimodale. Cette caractéristique lui permet de croiser simultanément et avec une grande précision des données hétérogènes (description textuelle des symptômes, radiographies, ordonnances numérisées), s'intégrant parfaitement à notre pipeline RAG.

- **Analyse Clinique Multimodale** : Concrètement, le système exploite ces capacités avancées pour analyser non seulement le texte saisi par l'utilisateur, mais également des fichiers médicaux complexes tels que des ordonnances, des radiographies ou des résultats d'analyses (PDF et images), le tout en une seule passe.
- **Génération en continu (Streaming)** : Pour offrir une expérience utilisateur fluide, semblable à une véritable consultation, les réponses de l'IA sont transmises en temps réel grâce à un flux de données asynchrone (Server-Sent Events).

- **Recherche Vectorielle Avancée** : Notre implémentation RAG s'appuie sur la base de données vectorielle ChromaDB. Les connaissances sont judicieusement réparties entre des collections distinctes (maladies communes et maladies rares).
- **Multi-Query et Reranking** : Pour pallier les ambiguïtés du langage patient, le système génère de multiples variantes de la requête initiale (Multi-Query Expansion) et applique un algorithme de Reranking basé sur la logique médicale pour réordonner les résultats vectoriels avant de les soumettre au LLM.

3.4.2. Sécurité et Validation Clinique

- **Moteur de Règles Cliniques** : Avant toute inférence par l'IA, les symptômes saisis sont analysés par un système de règles strictes détectant automatiquement trois niveaux d'urgence (critique, modéré, faible) en se basant sur des mots-clés spécifiques (ex: douleurs thoraciques, perte de conscience).
- **Validation Externe (API NIH)** : Afin de limiter les hallucinations du LLM, chaque pathologie envisagée est validée de manière asynchrone en interrogeant l'API officielle du NIH (*National Institutes of Health*). Cette étape permet de confirmer l'existence de la maladie et de lui associer son code standardisé ICD-10.
- **Algorithme de Smart Scoring** : Le calcul de la probabilité finale des maladies repose sur un score composite. Cet algorithme intègre le score vectoriel, le taux de correspondance des symptômes (*Jaccard similarity*), le niveau d'urgence, et la validation du NIH pour générer un pourcentage de confiance précis.

3.4.3. Orientation Médicale Géolocalisée

- **Recommandation intelligente** : Le système ne se limite pas à proposer un diagnostic provisoire ; il assure également la continuité des soins. À l'issue de l'analyse, le moteur IA identifie la spécialité médicale requise. En croisant cette donnée avec l'emplacement géographique du profil patient (ville ou wilaya), le système interroge dynamiquement la base de données relationnelle pour recommander les praticiens pertinents et les mieux notés, facilitant ainsi la prise de rendez-vous immédiate.

3.4.4. Optimisation des Performances et Apprentissage

- **Cache Intelligent** : Un mécanisme de mémorisation basé sur le hachage (SHA-256) et la similarité textuelle a été développé pour intercepter les requêtes fréquentes. Si un patient décrit des symptômes similaires à un diagnostic déjà évalué, le système renvoie la réponse en cache, réduisant ainsi drastiquement la latence.
- **Algorithme Génétique** : L'architecture intègre une boucle d'amélioration continue. Un algorithme génétique évalue différentes générations d'hyperparamètres (taux de mutation, taille de population) sur des données d'entraînement pré-établies afin d'optimiser mathématiquement la pertinence des réponses générées.

3.5. Présentation des Interfaces de la plateforme :

3.5.1 Tableau de bord principal de l'espace patient :

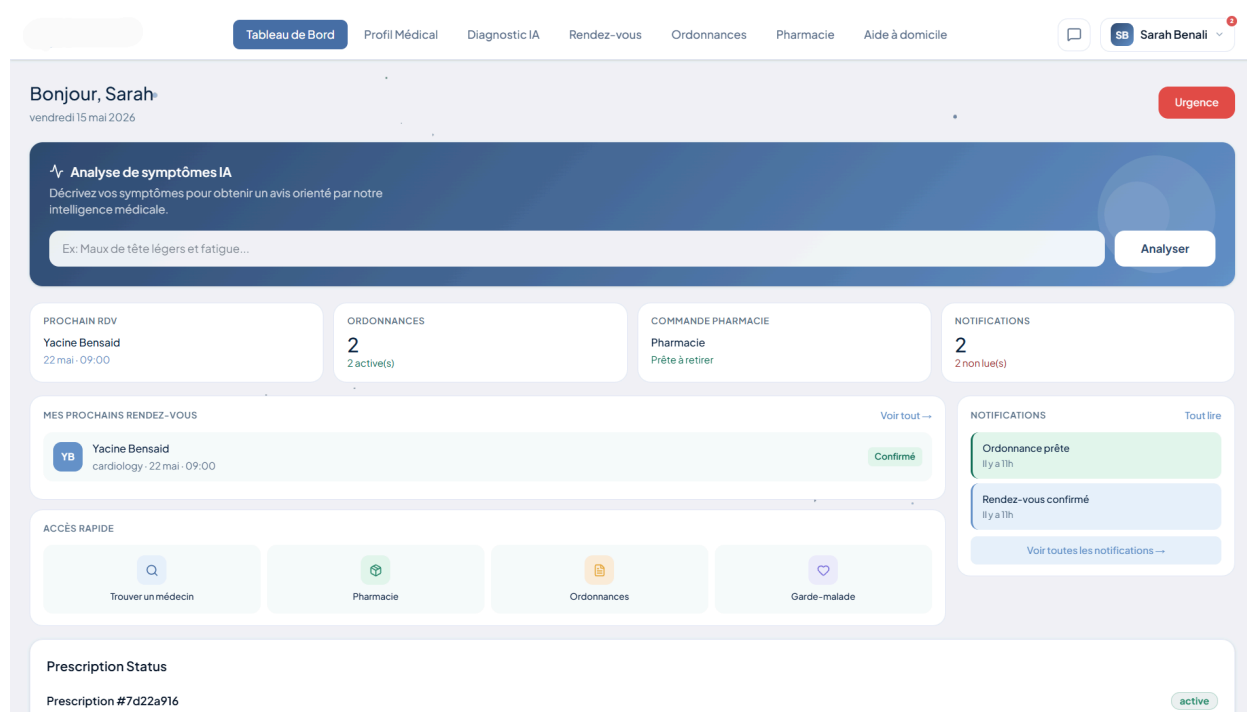


Figure 12 - Interface du Dashboard Patient

Cette interface constitue le point d'entrée principal du patient après son authentification. Conçue pour offrir une vue d'ensemble ergonomique, elle centralise les informations urgentes et les accès rapides. On y retrouve en tête de page l'accès direct au module d'analyse de symptômes par IA. Le tableau de bord affiche également des widgets dynamiques récapitulant les prochains rendez-vous confirmés, le statut des ordonnances actives, ainsi qu'un centre de notifications en temps réel pour le suivi des commandes en pharmacie.

3.5.2 Le Profil Médical Centralisé :

L'onglet Profil Médical matérialise la dématérialisation du dossier patient. Il regroupe les données biométriques (calcul automatique de l'IMC), les allergies connues et l'historique des antécédents médicaux (chroniques ou résolus). Comme détaillé lors de la conception de notre pipeline RAG, c'est précisément l'ensemble de ces données structurées qui est injecté dynamiquement dans le contexte de notre modèle de langage (LLM) afin de personnaliser et sécuriser les pré-diagnosics générés par l'Intelligence Artificielle.

The screenshot displays a web application interface for a patient's medical profile. At the top, a navigation bar includes links for 'Tableau de Bord', 'Profil Médical' (highlighted), 'Diagnostic IA', 'Rendez-vous', 'Ordonnances', 'Pharmacie', and 'Aide à domicile'. The user 'Sarah Benali' is logged in, with a 'Modifier' button. The profile header shows the patient's name, initials 'SB', age '34 ans', gender 'Féminin', and blood type 'A+'. The main content is divided into several sections: 'Informations personnelles' (phone, city, wilaya, emergency contact), 'Données médicales' (weight, height, BMI, allergies), 'Antécédents médicaux' (chronic and resolved conditions), 'Diagnostics récents', and 'Ordonnances'. Each section contains specific data and interactive elements like 'Voir détail' or 'PDF' buttons.

Informations personnelles	
TÉLÉPHONE	0555123456
VILLE	Alger Centre
WILAYA	Alger
CONTACT D'URGENCE	Mohamed Benali - 0555987654

Données médicales		
POIDS	TAILLE	IMC
62 kg	165 cm	22.8
ALLERGIES		
▲ Pénicilline ▲ Asprine		

Antécédents médicaux	
Hypertension artérielle	Chronique
Depuis 01/01/2018 - Personnel	
Fracture du poignet gauche	Résolu
Depuis 01/06/2015 - Personnel	

Diagnostics récents	
Rhinite allergique saisonnière	Voir détail
Dr. Amina Khelifi - 26/04/2025	

Ordonnances	
Loratadine	PDF
15/05/2026	

Figure 13 - Interface du Profil Médical du Patient

3.5.3 Le Module de Prise de Rendez-vous :

Cette interface illustre le système d'orientation médicale de la plateforme. Le patient dispose d'un moteur de recherche multicritères avancé (par spécialité, wilaya, disponibilité, évaluation et genre du médecin). Les résultats sont affichés sous forme de cartes synthétiques couplées à une carte interactive (Map) sur la droite, permettant une visualisation spatiale immédiate des cabinets médicaux pour faciliter le choix du praticien le plus proche.

Chapitre III : Réalisation et Implémentation

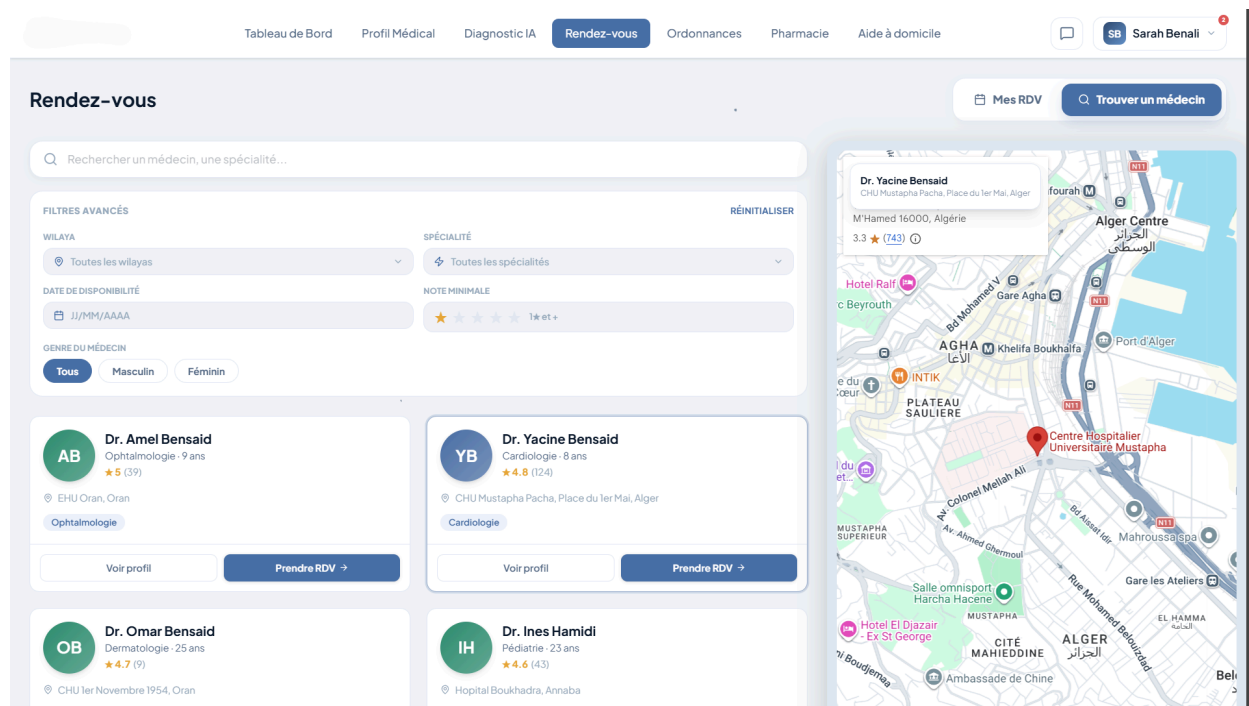


Figure 14 - Interface Prise de Rendez-vous Patient

3.5.4 Le Tableau de bord du Médecin :

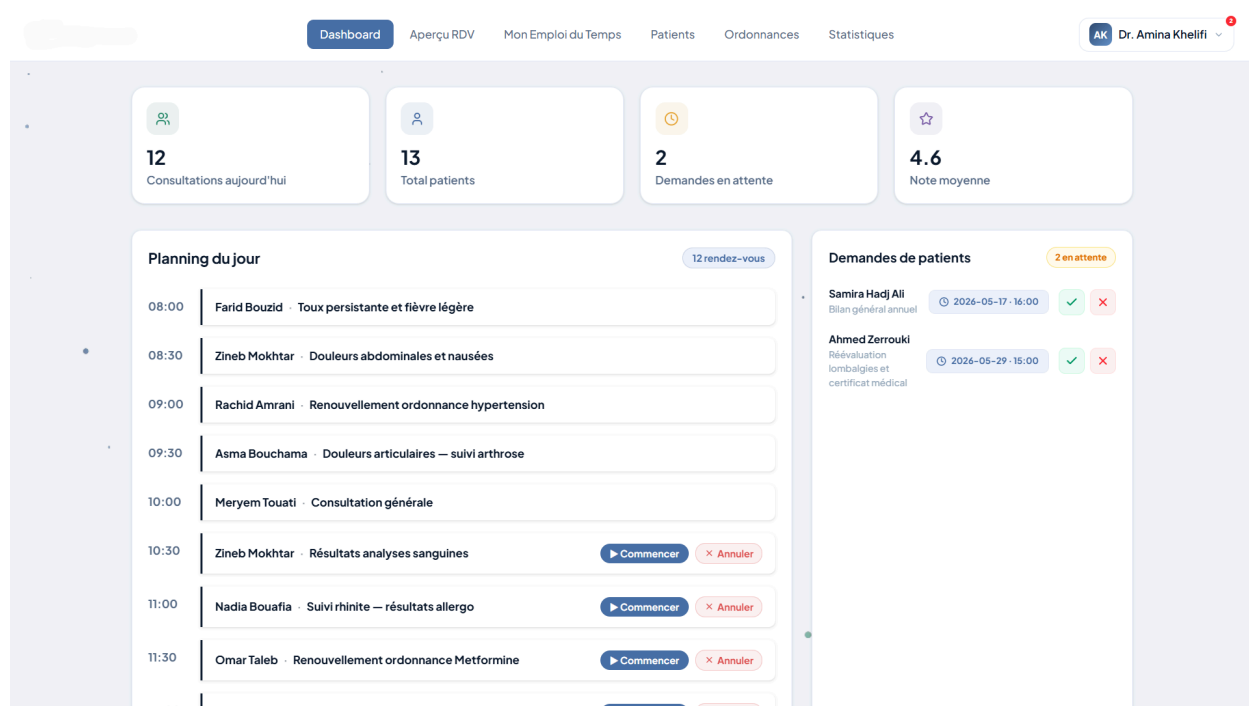


Figure 15 - Interface Dashboard Médecin

Ce tableau de bord centralise l'activité du médecin pour optimiser son flux de travail et réduire sa charge administrative. Il regroupe en une seule interface des indicateurs de performance en temps réel, un planning quotidien dynamique (incluant les motifs de consultation), ainsi qu'un module de gestion instantanée des demandes de rendez-vous.

3.5.5 Le Module d'Ordonnance Numérique :

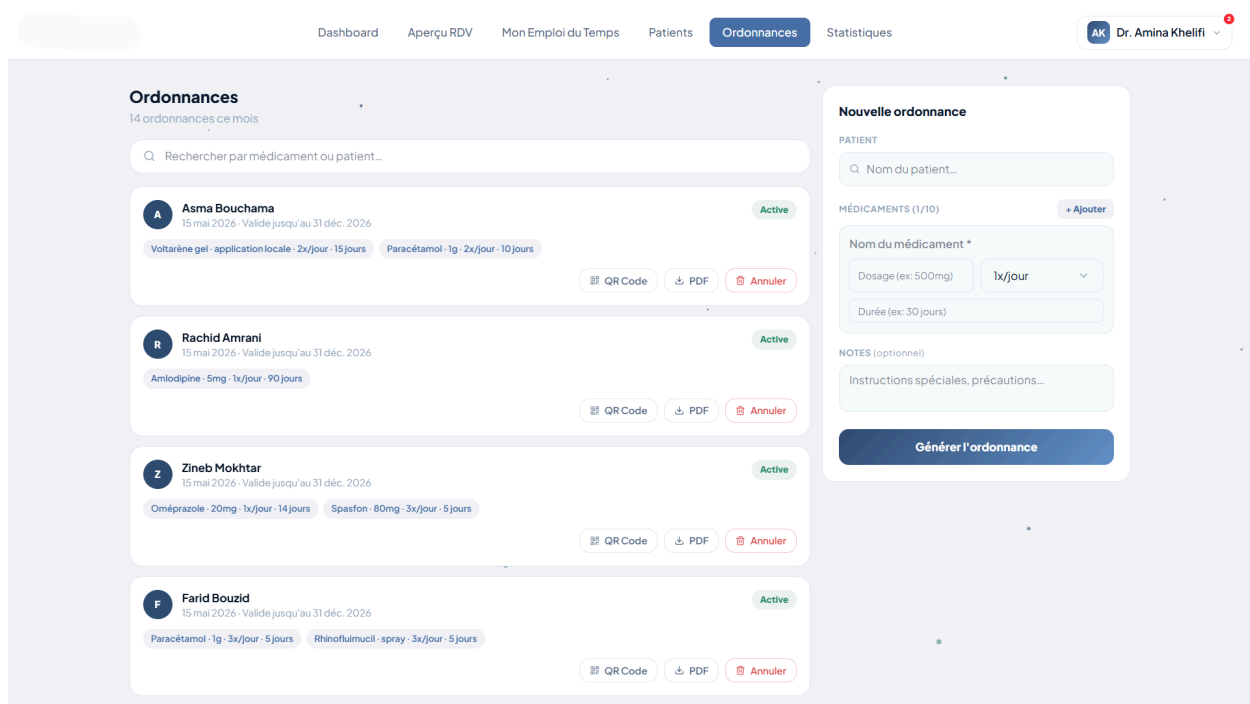


Figure 16 - Interface Création et Gestion d'Ordonnance Médecin

Ce module centralise la gestion des prescriptions médicales pour le praticien. Il combine un historique dynamique et filtrable des ordonnances (avec export PDF et QR Code) et un panneau latéral intégré pour la création de nouvelles prescriptions. Cette disposition ergonomique permet au médecin de rédiger la posologie et de générer la signature numérique tout en conservant une vue d'ensemble sur l'historique récent du patient.

3.5.6 Page du Diagnostic Médicale IA :

L'interface de ce module se compose d'un espace conversationnel à gauche, où le patient décrit ses symptômes par texte ou par commande vocale, et d'un panneau latéral à droite qui affiche en temps réel les résultats de l'analyse, incluant le niveau d'urgence, les hypothèses de pathologies estimées par l'IA et une liste de médecins disponibles à proximité pour faciliter la prise de rendez-vous immédiate.

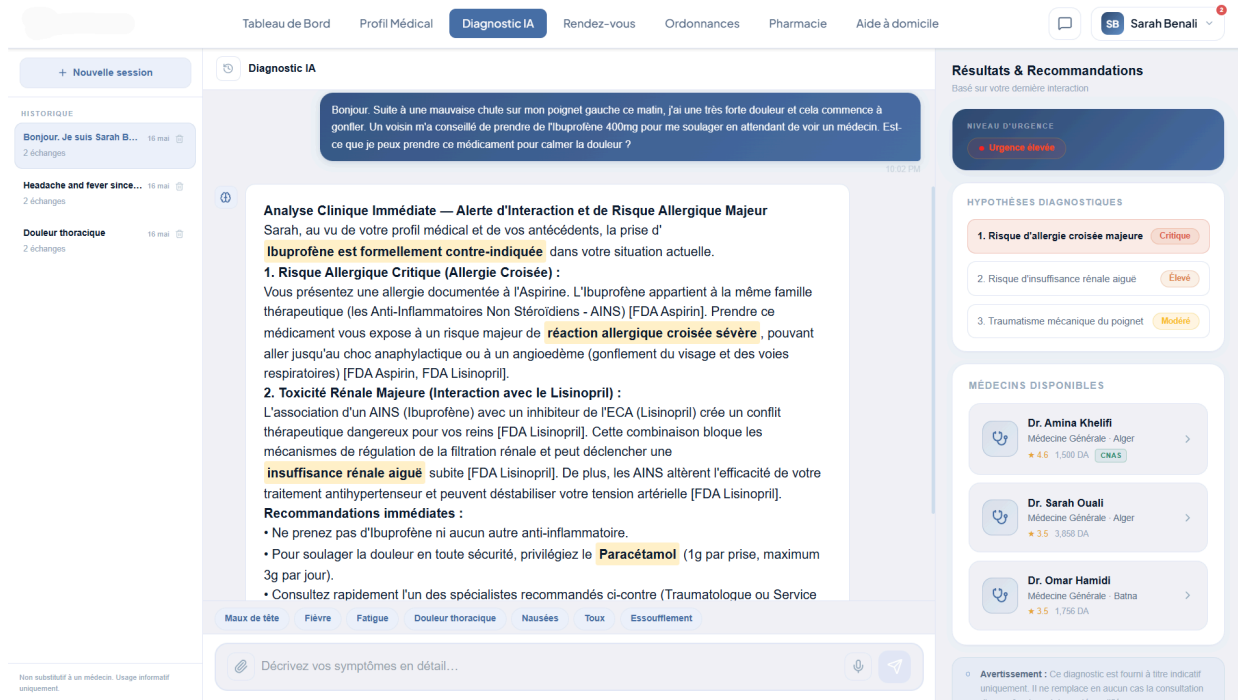


Figure 17 - Interface Diagnostic IA du Patient

3.6. Évaluation des performances et validation de l'architecture

3.6.1 Optimisation des temps de réponse (Impact du Cache SHA-256) :

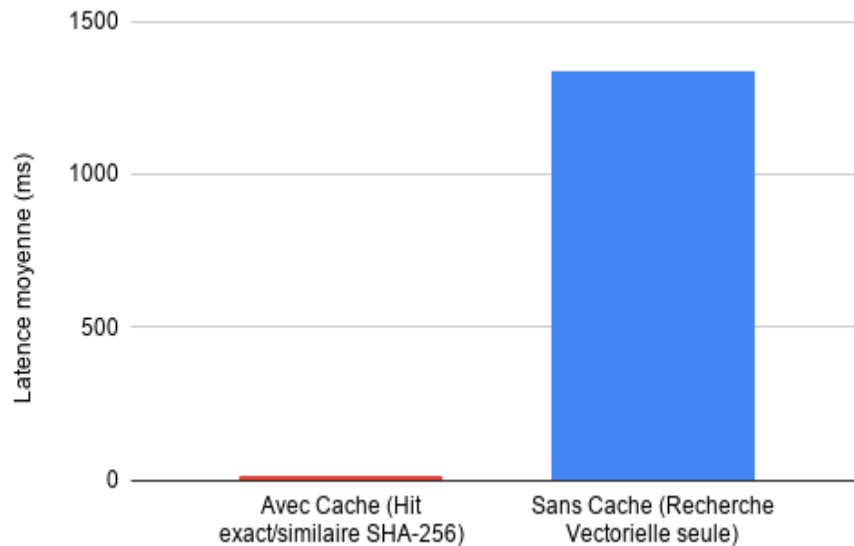


Figure 18 - Diagramme en barre horizontale de l'Impact du Cache Intelligent sur les performances

Ce graphique en barres (Figure 18) met en évidence l'impact radical de notre cache intelligent (SHA-256) sur les performances du système. On observe clairement qu'une recherche vectorielle complète (barre bleue) impose une latence moyenne de 1334 ms dans ChromaDB. En revanche, lorsque la requête est interceptée en situation de succès (« Cache HIT », barre rouge), ce délai s'effondre à seulement 16 ms, rendant l'exécution 83 fois plus rapide. Cette optimisation logicielle s'avère cruciale : elle garantit une excellente scalabilité à la plateforme, qui reste capable d'absorber de violents pics de connexions simultanées sans jamais ralentir l'expérience de l'utilisateur.

3.6.2 Optimisation des hyperparamètres par algorithme génétique :

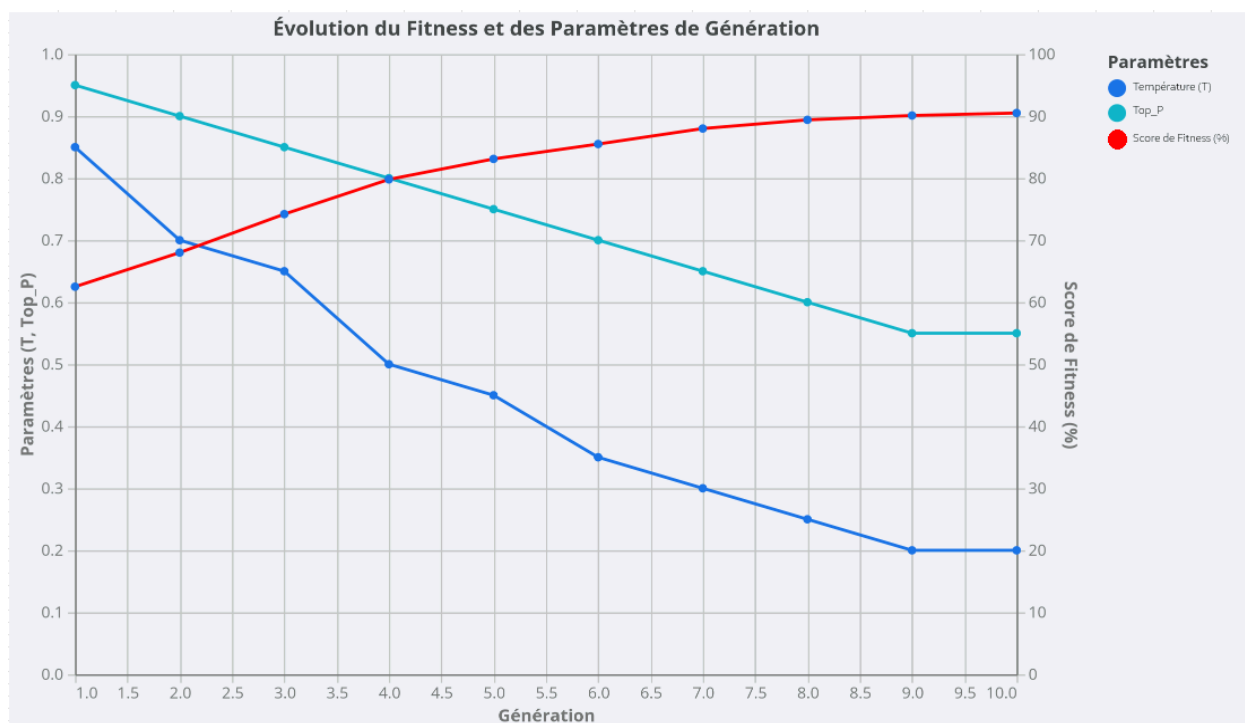


Figure 19 - Convergence des hyperparamètres et évolution du score de fitness via l'algorithme génétique

Ce graphique à double axe (Figure 19) illustre l'optimisation autonome du modèle LLM sur 10 générations. La courbe rouge (associée à l'axe secondaire) montre une progression constante du score de *fitness*, convergeant vers un optimum de 90,5 %. Simultanément, les courbes bleue et turquoise (axe principal) mettent en évidence la réduction progressive de la température (T) et du paramètre Top_P . Cette dynamique visuelle prouve que le système limite intentionnellement la créativité du modèle de langage pour maximiser la rigueur et la précision clinique des réponses.

3.6.3 Validation clinique du pipeline (Impact de l'API NIH) :

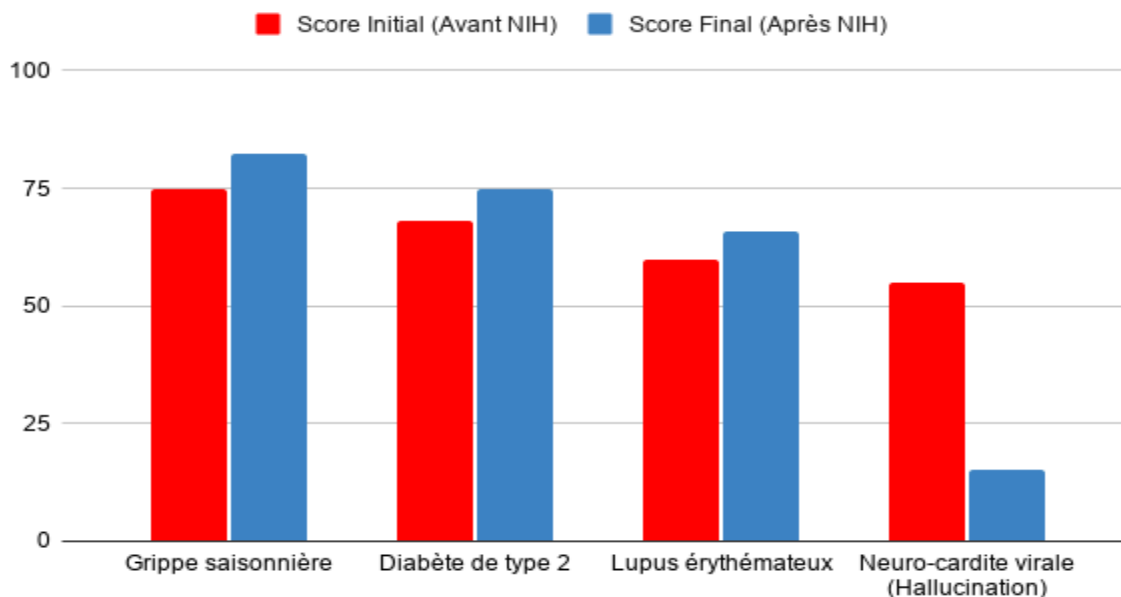


Figure 20 - Impact de la validation externe (API NIH) sur la sécurisation des pré-diagnosics

Cette figure (Figure 20) met en évidence le rôle indispensable de l'API du NIH pour sécuriser nos résultats. Concrètement, cette validation fait naturellement le tri : elle conforte d'abord les pathologies réelles (avec un gain de 7,5 points pour une grippe), mais agit surtout comme un bouclier contre les hallucinations de l'IA. Si le modèle invente une maladie fictive comme la "Neuro-cardite virale", le système fait immédiatement chuter son score de 55 à 15. Cela prouve que ce processus est capable de corriger de lui-même les fausses informations pour garantir la sécurité du patient.

3.6.4 Étude comparative globale : Architecture Healy (RAG) vs LLM classique

Ce graphique en radar (Figure 21 ci-après) illustre le compromis technique assumé de notre approche RAG. Comme attendu, le LLM standard conserve un avantage sur la "Généralité" ou la "Fluidité" conversationnelle, car il est naturellement conçu pour être créatif. Cependant, dans le domaine médical, cette liberté d'improvisation devient un risque d'hallucination inacceptable. C'est pourquoi le Bot Healy accepte de sacrifier un peu de cette souplesse pour exceller là où c'est vital : il surpasse massivement son concurrent sur la sécurité du patient, la précision médicale et la cohérence clinique.

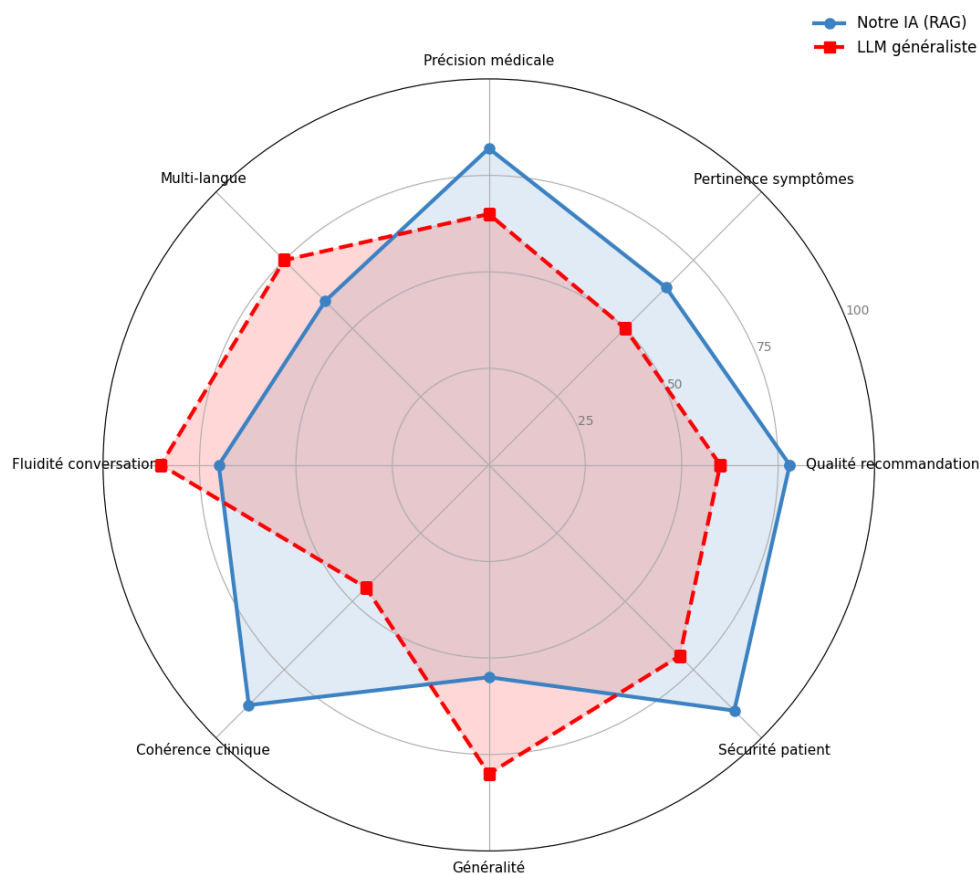


Figure 21 - Analyse comparative globale (Architecture RAG vs LLM standard)

En conclusion, cette évaluation confirme le succès de notre architecture. L'intégration du RAG et de l'API NIH a permis de canaliser un modèle de langage générique, souvent imprévisible, pour le transformer en un véritable dispositif d'assistance médicale spécialisé, fiable et sécurisé.

4. Conclusion Générale

En conclusion, ce projet démontre la faisabilité et l'efficacité de Healy, une plateforme e-Santé intelligente qui unifie le parcours de soins entre patients, médecins, pharmaciens et gardes-malades. Reposant sur une architecture client-serveur sécurisée (React/Django/PostgreSQL), elle intègre un assistant de pré-diagnostic IA optimisé par un pipeline RAG et validé en temps réel par l'API du NIH, éliminant ainsi tout risque d'hallucination clinique. En offrant des interfaces dédiées à chaque profil (dossier médical connecté, ordonnances QR Code, gestion des stocks et des soins), Healy rationalise la charge administrative et sécurise le circuit médical. À terme, l'enrichissement continu de sa base vectorielle et l'intégration d'objets connectés (IoT) ouvriront la voie à une véritable médecine 4.0, performante, humaine et connectée.

Webographie et Bibliographie

- [W1] **World Health Organization (WHO)**. Digital Health. Disponible sur :
<https://www.who.int/health-topics/digital-health>
- [W2] **Google DeepMind**. Gemini AI Models. Disponible sur :
<https://deepmind.google/technologies/gemini/>
- [W3] **Hugging Face**. multilingual-e5-large Model. Disponible sur :
<https://huggingface.co/intfloat/multilingual-e5-large>
- [W4] **ChromaDB**. ChromaDB Documentation. Disponible sur :
<https://docs.trychroma.com/>
- [W5] **LangChain. Retrieval-Augmented Generation (RAG)**. Disponible sur :
<https://python.langchain.com/docs/concepts/rag/>
- [W6] **National Library of Medicine (NLM)**. APIs and Developer Resources. Disponible sur :
<https://www.nlm.nih.gov/apis/>
- [W7] **World Health Organization (WHO)**. ICD-10 Classification. Disponible sur :
<https://icd.who.int/en>
- [W8] **Django Software Foundation**. Django Documentation. Disponible sur :
<https://docs.djangoproject.com/>
- [W9] **Django REST Framework**. Documentation officielle. Disponible sur :
<https://www.django-rest-framework.org/>
- [W10] **Meta**. React Documentation. Disponible sur :
<https://react.dev/>
- [W11] **PostgreSQL**. PostgreSQL Documentation. Disponible sur :
<https://www.postgresql.org/docs/>
- [W12] **JWT**. JSON Web Token Introduction. Disponible sur :
<https://jwt.io/introduction>
- [W13] **Microsoft**. Visual Studio Code Documentation. Disponible sur :
<https://code.visualstudio.com/>
- [W14] **Postman**. API Platform Documentation. Disponible sur :
<https://www.postman.com/>
- [W15] **GitHub**. GitHub Documentation. Disponible sur :
<https://github.com/>
- [W16] **Mermaid.js**. Mermaid Documentation. Disponible sur :
<https://mermaid.js.org/>
- [W17] **Visual Paradigm**. UML Tools Documentation. Disponible sur :
<https://www.visual-paradigm.com/>
- [W18] **OWASP Foundation**. Web Application Security. Disponible sur :
<https://owasp.org/>
- [W19] **Sihati24.dz**.
-

[W20] **Les grands modèles de langage (LLM)** - définition, fonctionnement et cas d'usage. IBM.

<https://www.ibm.com/fr-fr/think/topics/large-language-models>

[W21] **Ngrok** - outil de tunneling pour exposer un serveur local sur internet. Ngrok.

<https://ngrok.com/docs/start>

[W22] **Ministère de l'Industrie et de la Production Pharmaceutique (MIPH)**. Nomenclature Nationale des Produits Pharmaceutiques. Disponible sur :

<https://www.miph.gov.dz/fr/nomenclature-nationale-des-produits-pharmaceutiques/>

[W23] **Expo**. Expo Documentation. Disponible sur :

<https://expo.dev/>

[W24] **React Native**. React Native Documentation. Disponible sur :

<https://reactnative.dev/>

[W25] **Kaggle**. Kaggle.com. Disponible sur :

<https://www.kaggle.com/datasets/itachi9604/disease-symptom-description-dataset>

<https://www.kaggle.com/datasets/ulrich05/symptomes-et-maladie>

<https://www.kaggle.com/datasets/herviavalentine/symptomes>

<https://www.kaggle.com/datasets/ulrich05/symptomesxdiseases>

[B1] **BRAHIM Abderrazak et NADIL Aïcha Marwa. (2024).**

« Protection de la vie privée dans une plateforme E-santé ».

Université des Sciences et de la Technologie Houari Boumediene (USTHB).

[B2] **LARABI I ,SEKFALI R ,FEGAS L&OULDRABAH L (2025)** Réalisation d'un restaurant connecté utilisant un système de recommandation en temps réel pour les clients [Mémoire de licence]. Université des Sciences et de la Technologie Houari Boumediene.

[B3] **AMERICA, Pierre. (2023).** "Privacy and Security in E-Health Systems". ISTE Editions.

[B4] **ESTEVA, Andre, et al. (2019).** "A guide to deep learning in healthcare". Nature Medicine, 25(1), 24-29.

[B5] **World Health Organization (WHO).** (2021). "Global Strategy on Digital Health 2020-2025". Geneva: World Health Organization. Available at:

<https://www.who.int/docs/default-source/documents/gd4dhdaa2a9f352b0445bafbc79ca799dce4d.pdf>

[B6] **ISIL 089/24 (2024)** développement d'une plateforme de gestion de la santé en ligne pour le recherche des médecins et des médicaments.[Mémoire de licence]. Université des Sciences et de la Technologie Houari Boumediene (USTHB).

[B7] **ISIL 054/24 (2024)** Conception et Réalisation d'une Plateforme Intégrée pour la Gestion des Ordonnances électroniques. Université des Sciences et de la Technologie Houari Boumediene (USTHB).

[B8] **ISIL 074/24 (2024)** Réalisation d'un chatbot pour les clients de DJEZZY. Université des Sciences et de la Technologie Houari Boumediene (USTHB).

- [B9]Dr. Lamia BERKANI** COURS : INTRODUCTION À L'INTELLIGENCE ARTIFICIELLE Université des Sciences et de la Technologie Houari Boumediene (USTHB).
- [B10]Dr. Samira Oulefki** COURS : GÉNIE LOGICIEL Université des Sciences et de la Technologie Houari Boumediene (USTHB).
-

ANNEXE A

GLOSSAIRE ET DÉFINITIONS TECHNIQUES

A.1. Concepts Médicaux et Standards

- **ICD-10 (International Classification of Diseases, 10th Revision)** : Classification internationale des maladies publiée par l'Organisation Mondiale de la Santé (OMS). Elle attribue un code alphanumérique unique à chaque maladie, symptôme ou cause de blessure, permettant une standardisation mondiale des diagnostics.
- **API NIH (National Institutes of Health)** : Interface de programmation mise à disposition par l'institution gouvernementale américaine de recherche médicale. Elle permet d'interroger des bases de données de santé officielles et validées cliniquement.

A.2. Algorithmique et Intelligence Artificielle

- **Similarité de Jaccard (Jaccard Similarity)** : Indice statistique utilisé pour comparer la similarité et la diversité d'ensembles d'échantillons. Dans le contexte médical, il mesure le chevauchement exact entre les mots des symptômes décrits par le patient et les symptômes caractéristiques d'une pathologie.
 - **Similarité Cosinus** : Métrique mathématique mesurant l'angle entre deux vecteurs dans un espace multidimensionnel. Dans un système RAG, elle permet d'évaluer la proximité sémantique entre la requête du patient et une fiche maladie, indépendamment de la longueur des textes.
 - **Algorithme Génétique** : Algorithme d'optimisation inspiré de la théorie de l'évolution de Darwin. Il utilise des mécanismes tels que la sélection, la mutation et le croisement pour faire évoluer une population de solutions (ici, les hyperparamètres de l'IA) vers un résultat optimal (mesuré par le score de *Fitness*).
 - **Server-Sent Events (SSE)** : Technologie web permettant à un navigateur de recevoir des mises à jour automatiques à partir d'un serveur via une connexion HTTP unidirectionnelle. Elle est utilisée pour diffuser les réponses du modèle de langage (Streaming) en temps réel, token par token.
-

A.3. Architecture Logicielle et Base de Données

- **ORM (Object-Relational Mapping)** : Technique de programmation qui permet de manipuler les données d'une base de données relationnelle en utilisant des objets dans le code (comme avec Python/Django) au lieu d'écrire directement des requêtes SQL.
- **Architecture REST (Representational State Transfer)** : Style d'architecture logicielle définissant un ensemble de contraintes pour la création de services web. Une API RESTful utilise les méthodes HTTP standards (GET, POST, PUT, DELETE) pour interagir avec les ressources du système de manière sans état (*stateless*).
- **MVT (Modèle-Vue-Template)** : Patron de conception spécifique au framework Django. Le *Modèle* gère l'accès aux données, la *Vue* contient la logique métier en interceptant les requêtes HTTP, et le *Template* s'occupe de la présentation (l'interface utilisateur).

A.4. Sécurité et Cryptographie

- **Token JWT (JSON Web Token)** : Standard ouvert permettant d'échanger des informations de manière sécurisée sous forme d'objet JSON. Utilisé principalement pour l'authentification, il permet au serveur de vérifier l'identité de l'utilisateur sans avoir à stocker de session en base de données.
 - **PBKDF2 (Password-Based Key Derivation Function 2)** : Algorithme de dérivation de clé conçu pour sécuriser les mots de passe. Il applique une fonction pseudo-aléatoire (comme un hachage) au mot de passe avec une valeur de "sel" et répète le processus des milliers de fois pour ralentir les attaques par force brute.
 - **Hachage SHA-256 (Secure Hash Algorithm)** : Fonction de hachage cryptographique qui transforme une donnée de taille arbitraire en une empreinte numérique de taille fixe (256 bits). Dans cette plateforme, il est utilisé pour identifier rapidement si une requête médicale a déjà été traitée (mise en cache)
-

ANNEXE B

CONCEPTION SECONDAIRE

B.1. Définition de UML

UML (Unified Modeling Language) est un langage de modélisation standardisé utilisé pour visualiser, spécifier, construire et documenter les artefacts d'un système logiciel. Il propose une variété de diagrammes (cas d'utilisation, classes, séquences, etc.)

pour représenter la structure et le comportement du système. Ses avantages incluent une communication facilitée entre les parties prenantes, une réduction des ambiguïtés grâce à une notation unifiée, et une base solide pour la conception et la maintenance du logiciel.

B.2 Diagrammes de Classes

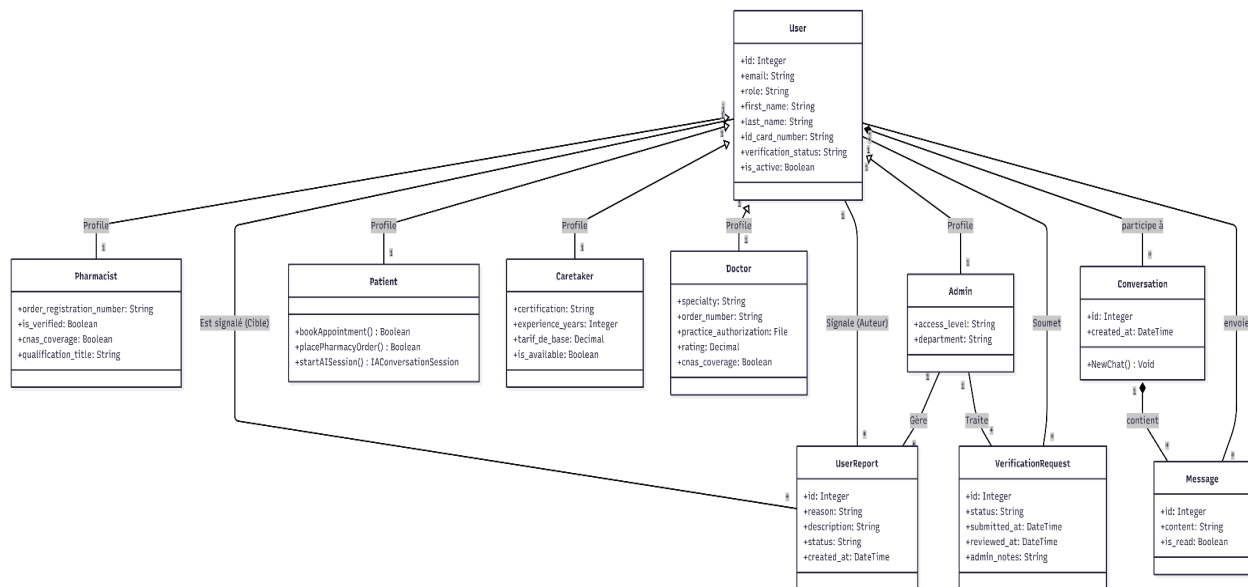


Diagramme de classes “user et rôles “

Il gère l'entité centrale User et la gestion des rôles via un système d'héritage . Il inclut la logique de messagerie sécurisée (Conversation, Message) permettant les échanges entre acteurs. Le module Admin assure la modération des inscriptions et l'audit du système via les journaux d'activité.

Annexe B : Conception Secondaire

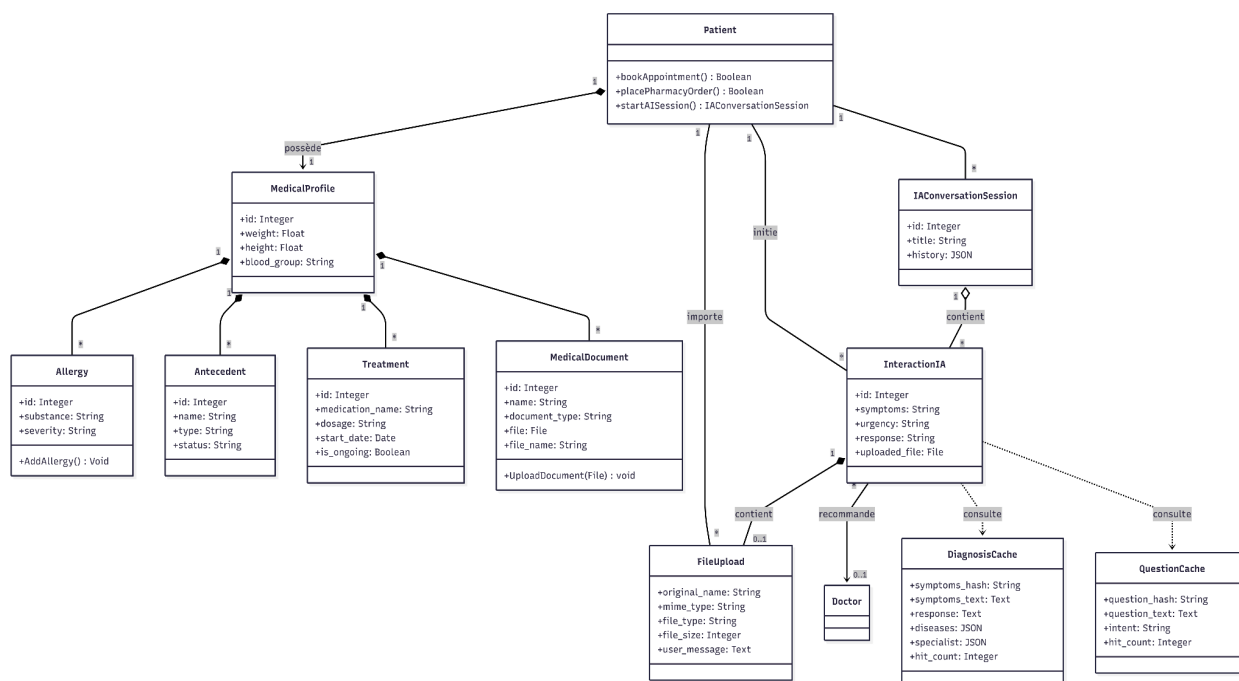
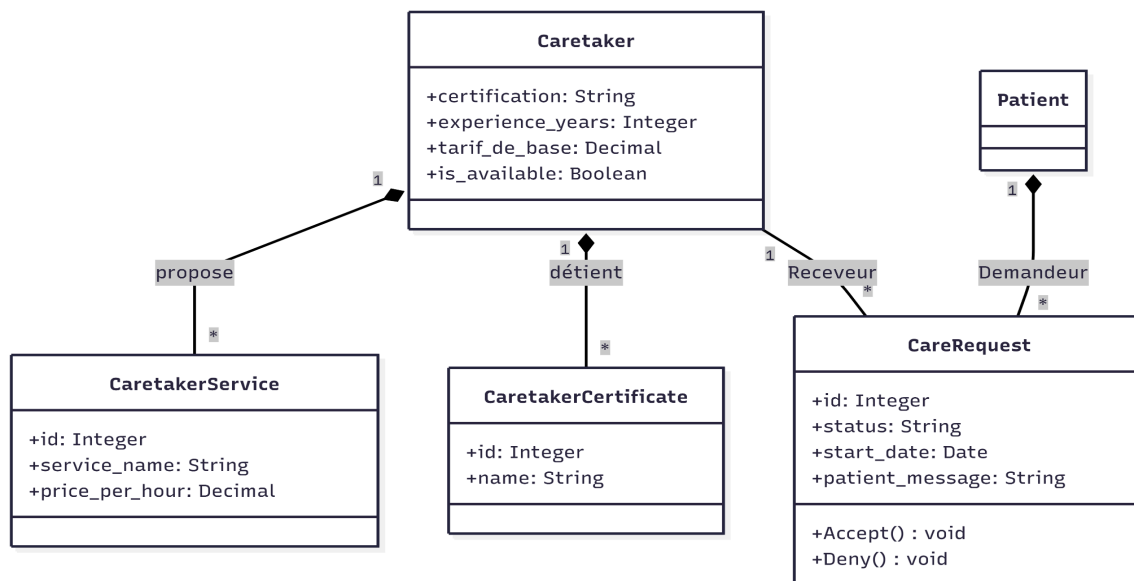


Diagramme de classes “patient et diagnostic AI “

Il s'articule autour de la classe Patient et de son MedicalProfile, regroupant les Allergy, Antecedent et MedicalDocument . La partie IA gère les AIConversationSession et les InteractionIA, stockant les analyses de symptômes (DiagnosisCache) et les résultats du pipeline RAG pour garantir la traçabilité des conseils médicaux .



Annexe B : Conception Secondaire

Diagramme de classes “caretaker “

Il repose sur la classe Caretaker dont le profil inclut l'expérience et les certifications. Il gère le cycle de vie des CareRequest (demandes de soins), incluant le suivi du statut de la mission (en attente, confirmée ou refusée) entre le demandeur et le prestataire .

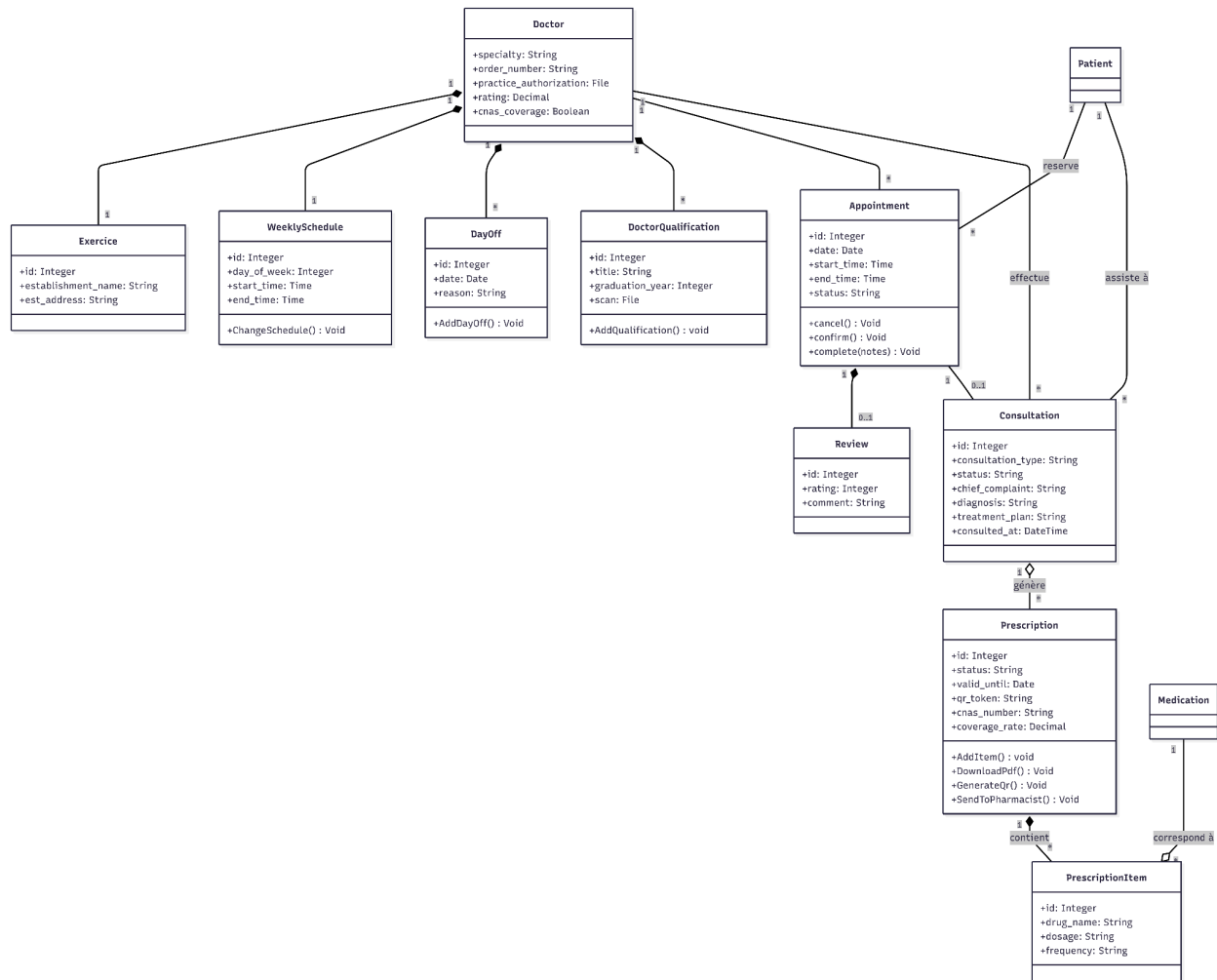


Diagramme de classes “doctor “

Il modélise les interactions entre le Doctor et le patient à travers les Appointment (rendez-vous) et les Consultation . Ce package inclut la gestion des Prescription numériques, décomposées en PrescriptionItem, et sécurisées par des QRToken pour la certification. Il intègre également un système de Review pour l'évaluation de la qualité des soins.

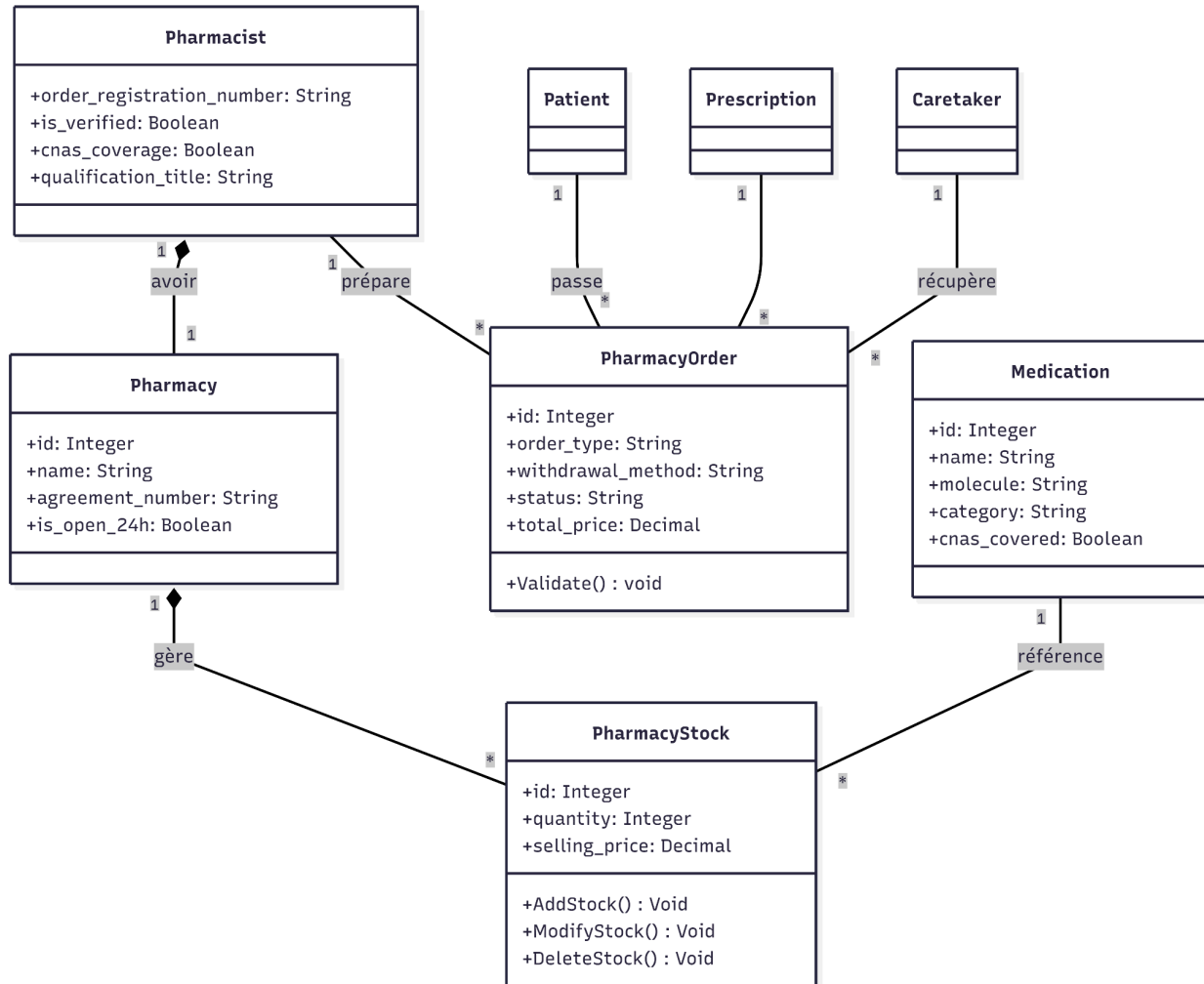


Diagramme de classes “pharmacy “

Il gère les officines via la classe Pharmacy rattachée au Pharmacist. Il inclut la nomenclature complète des Medication et la gestion en temps réel du PharmacyStock (quantités et prix). Le module traite les PharmacyOrder passées par les patients pour la réservation et l'achat de produits pharmaceutiques.

B.3 Diagrammes de Séquences:

B.3.1 Diagramme de Séquence du scénario “Ouvrir Session/Authentification”:

Ce diagramme détaille le flux de gestion des accès et de création de comptes sur la plateforme Healy. Il met en évidence le routage conditionnel basé sur le rôle de l'utilisateur (Patient ou Professionnel de santé). Lors de l'inscription, le système distingue l'activation immédiate d'un compte patient de la procédure stricte imposée aux professionnels de santé, qui nécessite la soumission de documents justificatifs et déclenche une vérification par un service de validation (mise en file d'attente). Lors de la connexion, le backend interroge le SGBD pour vérifier les

Annexe B : Conception Secondaire

identifiants et conditionne l'accès au tableau de bord des praticiens à la validation préalable de leur profil par l'administration.

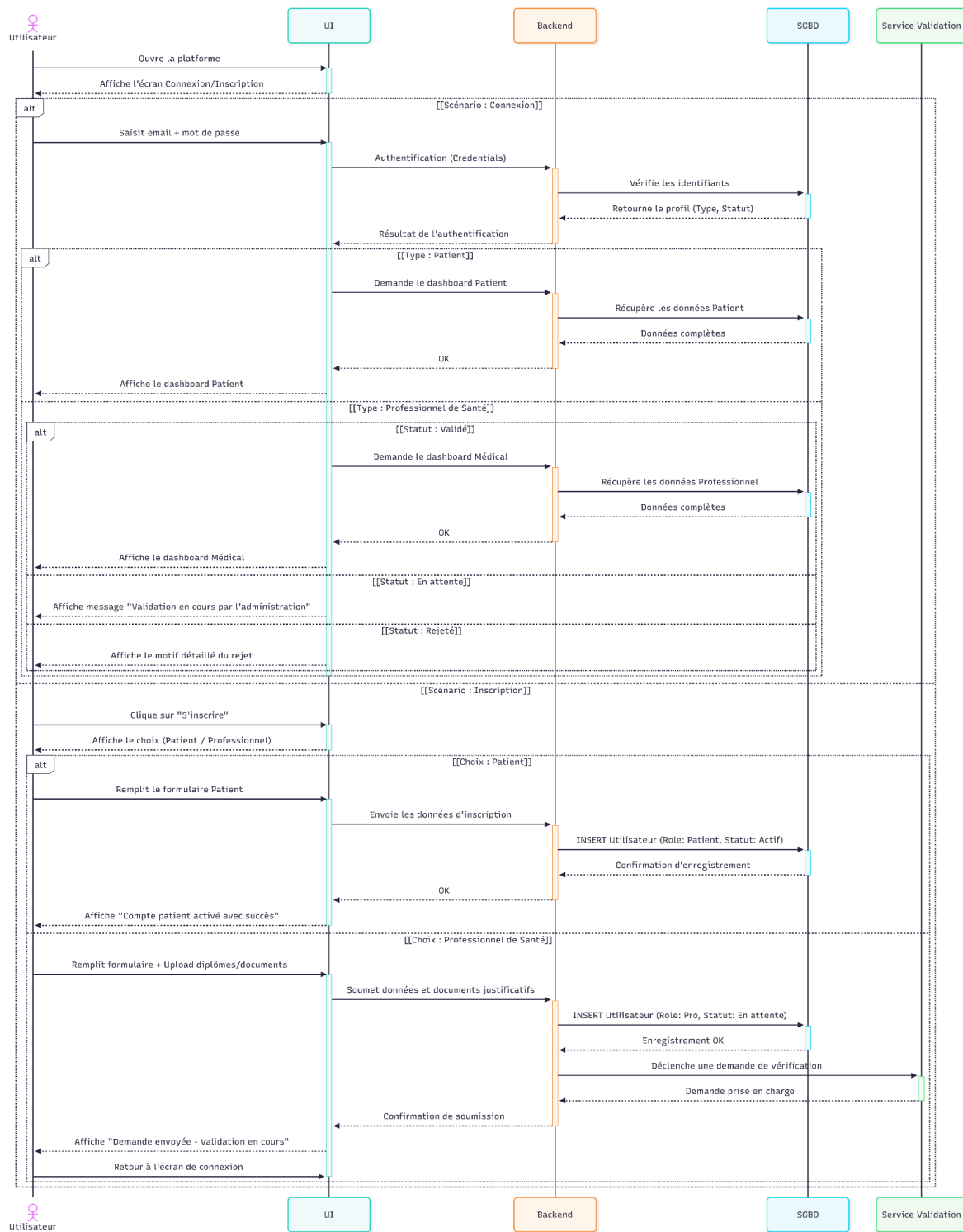


Diagramme de séquence du scénario "Ouvrir Session/Authentification"

B.3.2 Diagramme de Séquence du scénario “Traiter Demande Inscription”:

Ce diagramme illustre le processus administratif d'examen et de validation des inscriptions des professionnels de santé. L'interface sollicite le backend pour récupérer les profils en attente depuis le SGBD. Une étape cruciale du processus exige que l'administrateur consulte et examine attentivement les documents justificatifs fournis par le médecin (diplômes, agréments), explicitement téléchargés depuis le serveur. À l'issue de cette vérification, l'administrateur émet une décision (validation ou rejet). Le backend se charge alors de mettre à jour le statut dans la base de données et transmet directement une notification au praticien pour l'informer de l'issue de sa demande.

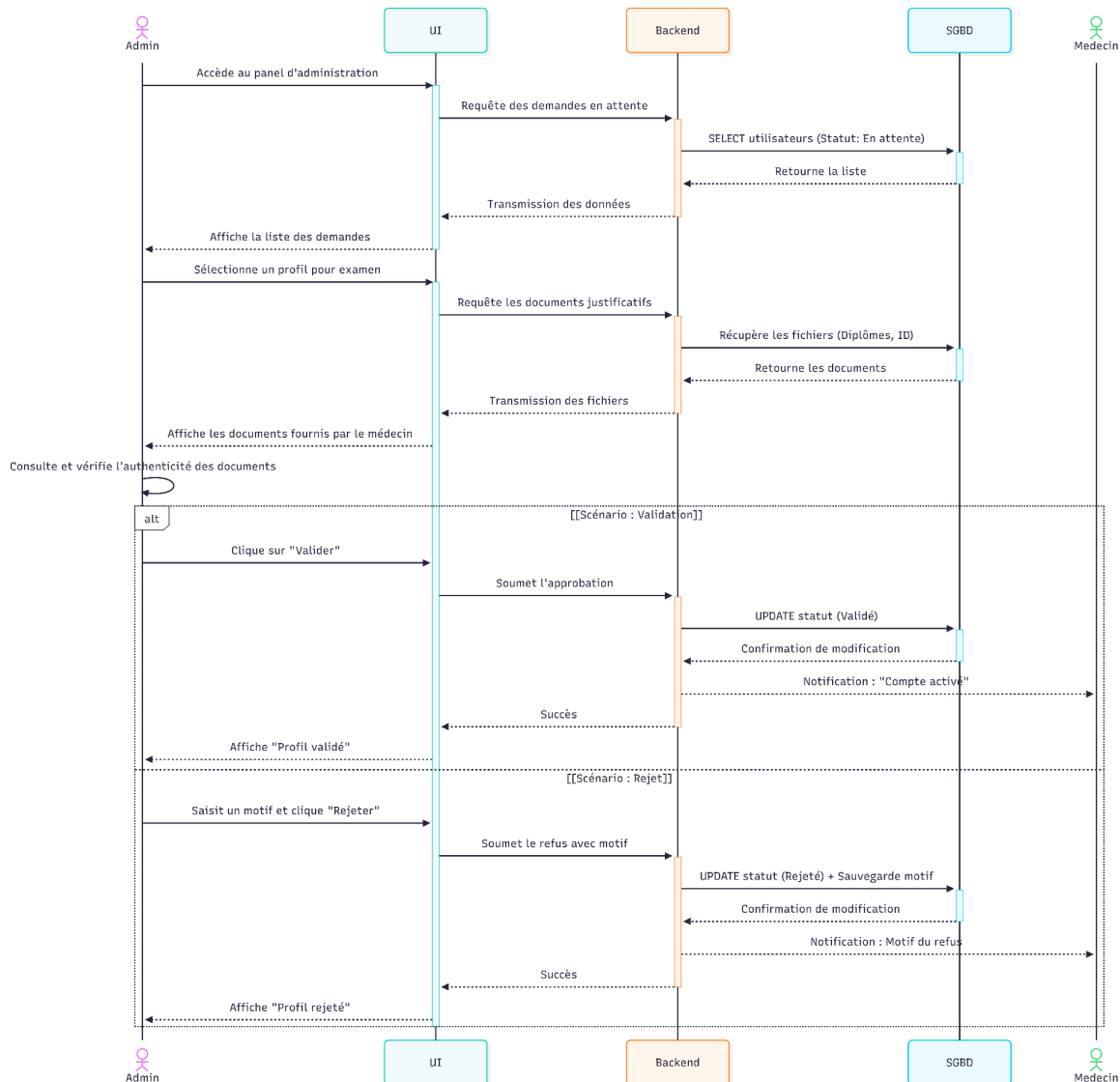


Diagramme de séquence du scénario “Traiter Demande Inscription”

ANNEXE C

Modèles de calcul et Résultats des Évaluations de l'Assistant IA

Cette annexe détaille les métriques, les modèles mathématiques et les résultats issus des tests automatisés de validation de notre système d'intelligence artificielle (Pipeline RAG). L'évaluation a été structurée autour de trois axes de performance.

C.1 Formules d'évaluation des performances

Pour quantifier les performances des différents modules, le système s'appuie sur le calcul de l'exactitude (*Accuracy*), qui mesure la proportion de prédictions correctes par rapport au nombre total de cas de test.

La formule générale de précision appliquée aux requêtes s'exprime ainsi :

$$Accuracy (\%) = \left(\frac{N_{correct}}{N_{total}} \right) \times 100$$

Où $N_{correct}$ représente le nombre de cas validés par le système, et N_{total} le nombre total d'échantillons testés.

Pour les scores globaux, le système utilise une moyenne arithmétique des sous-composants :

$$Score_{Global} = \frac{\sum_{i=1}^n Score_i}{n}$$

C.2 Qualité de la base de données (ChromaDB)

L'évaluation de la qualité des données s'est basée sur un échantillonnage des bases de connaissances médicales :

- **Collection** medical_common (2 332 documents) : Score de 96.6%
- **Collection** medical_rare (2 450 documents) : Score de 91.2%

Le score global de qualité des données ($Score_{data}$) est la moyenne arithmétique de ces deux collections:

$$Score_{data} = \frac{96.6 + 91.2}{2} = 93.9\%$$

C.3 Pertinence de la recherche vectorielle

Les tests évaluant la capacité du modèle d'embedding (sans l'intervention du cache) à retrouver le bon diagnostic ont démontré une fiabilité parfaite. Sur un ensemble de requêtes spécifiques, le taux de réussite (Top-1) a été calculé comme suit:

$$Score_{recherche} = \left(\frac{8}{8}\right) \times 100 = 100.0\%$$

Latence de recherche moyenne : 1 334 ms.

C.4 Performances du Bot (Pipeline RAG complet)

L'évaluation finale évalue le système en conditions réelles. Les scores par catégorie sont les suivants :

- **Détection de maladies :** $\left(\frac{13}{13}\right) \times 100 = 100.0\%$
- **Niveau d'urgence :** $\left(\frac{12}{13}\right) \times 100 \approx 92.3\%$
- **Cas limites gérés :** $\left(\frac{1}{2}\right) \times 100 = 50.0\%$

Le score consolidé attribué au comportement de l'assistant IA ($Score_{bot}$) s'établit à **90.2%**. Par ailleurs, l'optimisation par la mise en cache a permis de faire chuter la latence de réponse à une moyenne de 16 ms.

C.5 Bilan du Système Global

Le score d'évaluation final du système ($Score_{system}$) combine de manière pondérée la qualité des données, la fiabilité du moteur vectoriel et les performances globales du bot:

$$Score_{system} = \frac{Score_{data} + Score_{recherche} + Score_{bot}}{3} \approx 94.6\%$$

Conclusion : Avec un score global de **94.6%**, la plateforme satisfait largement les seuils de tolérance requis. Elle est par conséquent considérée comme pleinement opérationnelle et apte au déploiement en production.
